
A PRACTITIONER'S HANDBOOK

The Money Path

Designing, securing and auditing a Stripe integration that deserves to run in production.

18 chapters on architecture, compliance and failure modes

6 appendices including a full reference implementation

72 review points for auditing an integration you did not write

Ismaël Ladjounlou

Engineer and Technical Consultant

THE MONEY PATH

Designing, securing and auditing a Stripe integration that deserves to run in production.
First edition, 2026.

AUTHOR

Ismaël Ladjohounlou, engineer and technical consultant.

SCOPE AND LIMITS

This book describes Stripe's platform behaviour, the PCI DSS framework and European payment regulation as documented at the time of writing. Payment platforms move quickly. Product availability, thresholds, API versions and regulatory deadlines change. Every normative statement in this book should be checked against the current primary source before it is relied on commercially. Appendix F lists every source used.

NOT LEGAL OR COMPLIANCE ADVICE

Nothing here constitutes legal advice, a compliance attestation or a substitute for a Qualified Security Assessor. Where a decision carries regulatory weight, involve your acquirer, your QSA and your counsel.

TRADEMARKS

Stripe, Radar, Connect, Checkout, Elements, Link, Terminal, Sigma and Issuing are trademarks of Stripe, Inc. Supabase, Next.js, Vercel, PostgreSQL and all other product names are the property of their respective owners. This book is an independent work and is not endorsed by, affiliated with or sponsored by Stripe, Inc.

A payment system does not fail loudly. It fails in a way that looks exactly like success, for months, until someone finally counts the money.

Contents

FRONT MATTER

How to read this book	7
-----------------------	---

PART ONE · THE GROUND RULES

1	The Shared Responsibility Model	10
2	Eligibility: Who Stripe Can and Cannot Serve	16
3	Choosing an Integration Shape	20

PART TWO · THE REGULATORY SURFACE

4	PCI DSS 4.0.1 in Practice	25
5	Strong Customer Authentication and 3D Secure	30
6	The Payment Page: TLS, CSP and E-Skimming	35

PART THREE · THE SERVER SIDE

7	Keys, Secrets and Blast Radius	41
8	The Money Path: Server-Authoritative Payments	47
9	Webhooks: The Load-Bearing Wall	53
10	Fulfilment and Data Consistency	61
11	Rate Limits, Errors and Resilience	67

PART FOUR · FRAUD, RISK AND DISPUTES

12	Radar and Transaction Risk	73
13	Disputes, Chargebacks and Evidence	77

PART FIVE · PLATFORMS AND MARKETPLACES

14	Connect: Account Types and Liability	82
----	--------------------------------------	----

PART SIX · OPERATING THE INTEGRATION

15	API Versioning and SDK Discipline	87
16	Environments, Sandboxes and Testing	90
17	The Go-Live Checklist	93

18	The Security Review Grid	96
----	--------------------------	----

APPENDICES

A	Test Cards and Simulated Outcomes	102
B	Content Security Policy Directives	105
C	Webhook Events That Matter	108
D	Reference Implementation	111
E	Glossary	121
F	Sources	124

How to read this book

Some bugs only exist in payment code. They raise no exception and never show up in an error budget: the checkout button works, the card is charged, the customer is happy and the dashboard is green. Meanwhile the server has believed something the browser told it, and the business is quietly losing money.

I have spent enough time reading other people's integrations to know that this class of defect is not caused by ignorance of cryptography or of payment networks. It is caused by a single missing habit: the habit of asking, at every step, *which party is making this assertion, and why should I believe them*. Stripe's own documentation is excellent, but it is organised as a reference for people who already know what they are looking for. This book is organised around the thing you are actually protecting, which is the path the money takes.

Who this is for

Engineers shipping their first paid product. Technical leads inheriting an integration someone else wrote. Consultants asked to review a codebase and say whether it is safe to put behind real money. Founders who need to understand what they are signing up for before they sign. The code examples assume TypeScript on a Node runtime, and the reference implementation in Appendix D targets Next.js with a PostgreSQL backend, but every principle in Parts One, Two, Four and Six is language independent.

How it is organised

Six parts, moving from the outside in. Part One establishes who is responsible for what, who is even allowed to use the platform, and which integration shape you should pick, because that single decision determines most of your compliance burden. Part Two covers the regulatory surface: PCI DSS, strong customer authentication, and the client-side attacks the standard was re-written to address. Part Three is the engineering core: keys, the server authoritative payment flow, webhooks, fulfilment and resilience. Part Four covers fraud and disputes. Part Five covers platforms. Part Six is operations, testing and the two checklists you will actually use.

The four marks in the margin

Four kinds of box appear throughout. They are the compressed version of the book.

RULE

A non-negotiable. If your integration violates a rule box, it is not ready for production, regardless of how well it tests.

TRAP

A pattern that looks correct, passes a manual test, and is wrong. These are the ones that reach production, because nothing about them feels like a mistake at the time.

PRACTICE

Something that costs a little now and saves a great deal later. Not mandatory, but the difference between an integration that works and one that is maintainable.

CONTEXT

Background, history or a platform detail that explains why something is the way it is.

On sources and on time

Everything normative in this book is traceable to a primary source listed in Appendix F: Stripe's documentation, Stripe's own published engineering guidance, the PCI Security Standards Council, and the text of the relevant European regulation. Where I give a number, a threshold or a limit, it was accurate when written and it will eventually stop being accurate. Thresholds move. API versions ship monthly. The PCI standard revises. Treat every figure in this book as something to verify, and treat the reasoning around it as the part with a longer shelf life.

PART ONE

The Ground Rules

Before a single line of integration code is written, three questions decide most of what follows: who is liable for what, whether you are allowed to transact at all, and which shape of integration you choose.

- 1 The Shared Responsibility Model
- 2 Eligibility: Who Stripe Can and Cannot Serve
- 3 Choosing an Integration Shape

CHAPTER ONE

The Shared Responsibility Model

Stripe protects the card data. What your own code does with the payment, and what it gives the customer afterwards, is still your responsibility.

When a company adopts Stripe, a comfortable belief tends to settle in: payments are now handled. Stripe is a PCI Level 1 certified service provider, the highest tier of the standard, audited annually by an external assessor. It maintains its own certification, encrypts card data at rest, isolates the decryption keys from the systems that hold the ciphertext, forces TLS on every connection and terminates card data inside infrastructure you never touch. All of that is true and none of it protects your revenue.

The reason is simple. Stripe is responsible for the *secrecy of the instrument*. You remain responsible for the *correctness of the transaction*. A card number that is perfectly protected can still be used to buy a one thousand dollar plan for one dollar, if your server took the price from the browser. A payment that genuinely succeeded can still fail to deliver the product, if your fulfilment logic never ran. A payment that never happened at all can still unlock a subscription, if your webhook endpoint believes anyone who posts to it.

Stripe removes an entire category of risk from your business. It does not remove the category you are most likely to get wrong.

1.1 Drawing the line precisely

The division of labour is not vague. It can be written down, and it should be, because teams routinely assume the wrong side owns a control. The table below is the version I use when opening a review.

CONTROL	OWNER	WHAT IT MEANS IN PRACTICE
Storage of the primary account number	Stripe	Card data is held in an isolated environment, encrypted at rest, with decryption keys kept on separate infrastructure from the encrypted values.
Transport encryption to the API	Stripe	HTTPS is enforced on every endpoint, with HSTS on the dashboard and site. Plaintext requests are refused, not downgraded.
PCI certification of the vault	Stripe	Annual audit as a Level 1 service provider. Their attestation covers their environment only.
Network authorisation and 3DS orchestration	Stripe	Routing to the issuer, challenge flows, exemption signalling where supported.
Fraud scoring infrastructure	Shared	Stripe provides the model and the signals. You choose the thresholds, the rules and what to do with a flagged payment.

CONTROL	OWNER	WHAT IT MEANS IN PRACTICE
Authentication of your own users	You	Stripe has no idea who is logged into your application. Binding a payment to a user is entirely your problem.
The amount charged	You	Stripe charges what your server asks it to charge. If your server asks for the wrong number, the charge is still valid.
Authenticity of inbound webhooks	You	Stripe signs every event. Verifying that signature is a step you must write and can silently skip.
Fulfilment: granting access, shipping, provisioning	You	Nothing in the platform knows what a successful payment entitles a customer to.
Secret key custody and rotation	You	Stripe issues the key. Everything after issuance is your operational discipline.
Integrity of your own payment page	You	Scripts you load, headers you set, dependencies you pull. This is where card skimming actually happens.
Dispute evidence	You	Stripe transmits what you submit. The quality of the evidence is yours to produce.

Table 1.1 The responsibility split. Every row on the "You" side is a place an integration can be correct or incorrect independently of Stripe.

CONTEXT

The phrase "Stripe is PCI compliant, so we are PCI compliant" is one of the most expensive misreadings in the industry. Stripe's certification reduces *your* scope, sometimes dramatically, but it never eliminates it. You still complete a self assessment questionnaire, and as of the 4.0 revision of the standard even the lightest questionnaire carries obligations about the scripts on your payment page. Chapter 4 works through exactly what remains yours.

1.2 The trust boundary

If you internalise one diagram from this book, make it this one. Every payment defect I have found in a code review reduces to a value crossing this line in the wrong direction, or crossing it without validation.

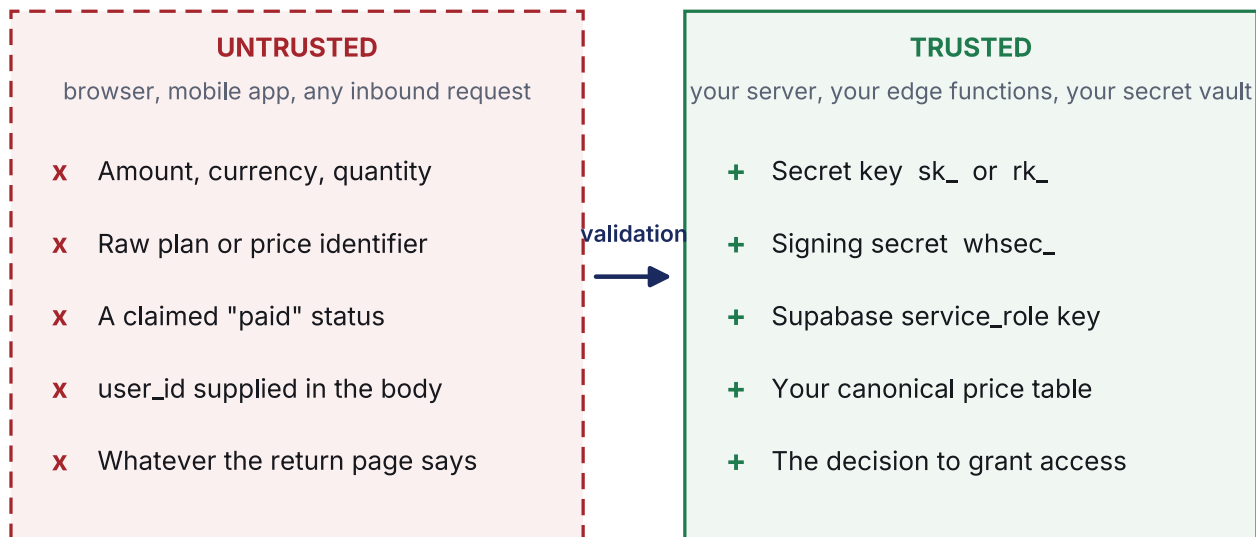


Figure 1.1 The trust boundary. Anything on the left is an assertion by a party with an incentive to lie. Anything on the right is either a secret you hold or a decision only you can make. Values may cross from left to right only through validation, and nothing on the right may ever cross to the left.

The left column is not a list of hostile inputs from a hostile user. It is a list of inputs from *any* user, including the honest majority, because the boundary is a structural property and not a judgement about intent. A browser is an environment the user fully controls. Every field in a request body, every query parameter, every header, every value in local storage is a claim the user could have written by hand. This is true even when your own JavaScript put it there, because your own JavaScript runs on their machine.

RULE

The client may tell you *what it wants*. It may never tell you *what it costs*, *who it is for*, or *whether it was paid*. Those three facts are resolved on the server, from data the server owns.

1.3 A threat model you can hold in your head

Formal threat modelling has its place, but for a payment integration you can get most of the value from five named adversaries. Each one has a distinct motive, which means each one attacks a different control.

The price manipulator

Wants your product for less than it costs. Opens developer tools, finds the request that creates the payment, changes `amount` or `quantity` or swaps a `price_id` for a cheaper one, and replays it. Costs nothing to attempt, requires no skill, and succeeds against a surprising share of integrations. Defeated by resolving price server side, covered in Chapter 8.

The card tester

Has a list of stolen card numbers and needs to know which ones still work. Your public checkout is a free oracle: a small charge that succeeds tells them the card is live. They do not want your

product, they want the boolean. This is why card testing shows up as a flood of low value attempts with high decline rates, why it corrodes your authorisation rate, and why it can trigger network fines. Defeated by rate limiting, Radar rules and never exposing a create-payment endpoint without an authenticated session. Chapter 12.

The webhook forger

Discovers your webhook URL, which is usually guessable and frequently published in a public repository, and posts a well formed `checkout.session.completed` event to it. If you do not verify the signature, you have handed out a free lifetime subscription endpoint. Chapter 9 is entirely about this adversary.

The replayer

Does not forge anything. Captures one genuine signed event, or simply exploits the fact that Stripe itself retries delivery, and gets the same value granted several times: credits added twice, a referral bonus paid five times, an order shipped repeatedly. Beaten only by idempotency, which is a property of your handler and not of the platform. Chapters 9 and 10.

The supply chain

Never touches your servers. Compromises a script your checkout page loads, an analytics tag, a chat widget, a transitively pulled npm package, and reads the card fields as the customer types them. This is the attack that rewrote the PCI standard in 2022 and made script inventory a hard requirement in 2025. Chapter 6.

Two observations about that list. First, four of the five adversaries never break cryptography, never find a memory corruption bug and never need a zero day. They exploit a missing check. Second, the platform defends against none of them on your behalf, because from Stripe's point of view every one of these requests is a valid, correctly authenticated call from a merchant who presumably knows what they are doing.

1.4 What actually goes wrong, ranked

Over a large enough sample of integrations, the same defects recur with striking consistency. The chart below plots them on two axes that matter more than a raw count: how bad it is when it happens, and how often you find it.

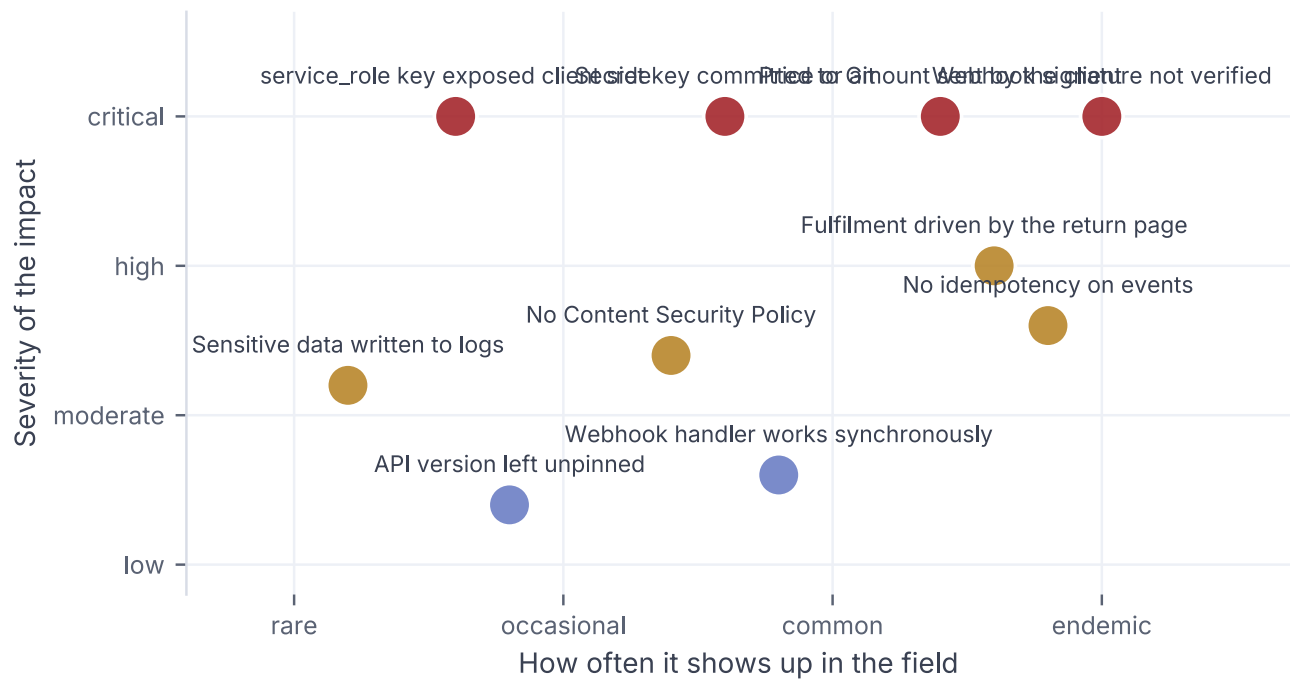


Figure 1.2 The recurring defects, positioned by severity of impact against how frequently they appear in production codebases. The upper right quadrant is where a security review should spend its first hour.

The concentration in the upper right is not an accident of my sample. An independent scan of several thousand publicly reachable applications with Stripe webhook endpoints found that roughly a quarter of them accepted unsigned events, meaning any third party who knew the URL could assert that a payment had occurred. That is not an exotic failure. That is the single most consequential control in the entire integration, absent from one endpoint in four.



Figure 1.3 Findings from a public scan of roughly six thousand applications exposing Stripe webhook endpoints. Around 1,542 performed no signature verification at all.

TRAP

Every one of these defects passes a manual test. You click through checkout with a test card, the payment succeeds, the subscription appears, the receipt arrives. Nothing in that exercise touches the signature check, the price resolution or the replay path. A payment integration that "works" has demonstrated only that the happy path works when nobody is attacking it.

1.5 The shape of the rest of this book

Everything that follows is an elaboration of Table 1.1 and Figure 1.1. Part Two takes the compliance rows and turns them into specific obligations. Part Three takes the engineering rows and turns them into code. Part Four takes the fraud row. Part Six turns all of it into two checklists you can run against a codebase.

Before any of that, though, there is a question that precedes design entirely, and that teams routinely discover the answer to at the worst possible moment: whether Stripe will serve your business at all.



CHAPTER TWO

Eligibility: Who Stripe Can and Cannot Serve

Check that Stripe will accept your business before you write any code. Some activities are restricted and some are refused outright, and finding out after launch means moving the payment layer somewhere else.

Payment processing is not a neutral utility. Stripe sits inside a regulated chain: card networks impose rules on acquirers, acquirers impose rules on payment facilitators, financial regulators impose rules on everyone, and sanctions regimes sit above all of it. When Stripe declines to serve a business, it is usually not a product decision. It is Stripe passing along a constraint it does not control either.

There are three gates. Geography, legal entity, and the nature of the activity itself.

2.1 Geographic availability

Stripe operates in a large but bounded set of countries, and the list expands over time. The critical detail, which trips up more teams than the list itself, is that availability is determined by where the *business* is legally established and banked, not by where the founder lives, where the servers run or where the customers are.

RULE

Your Stripe account country is the jurisdiction of your legal entity and its bank account. You can sell to customers almost anywhere. You cannot open an account from a country Stripe does not support merely because your users are elsewhere.

The practical consequences of that rule are worth spelling out, because founders in unsupported jurisdictions consistently encounter the same three options and the same three sets of trade-offs.

APPROACH	HOW IT WORKS	WHAT IT COSTS YOU
Incorporate in a supported country	Register a real legal entity (commonly in the United States, the United Kingdom or an EU member state) with a genuine local bank account, then onboard normally.	Formation and annual maintenance costs, local tax filing obligations, and a genuine obligation to operate that entity honestly. This is legitimate, and it is what most cross border SaaS businesses do.
Use a merchant of record	A third party such as a reseller platform becomes the legal seller. They handle payment, tax and compliance, and remit to you.	Materially higher fees, less control over the customer relationship, and dependence on their risk policies. In exchange you inherit their eligibility and their tax handling.

APPROACH	HOW IT WORKS	WHAT IT COSTS YOU
Use an alternative processor	Regional acquirers, mobile money aggregators or local PSPs that operate where you are.	Weaker developer tooling in most cases, but they serve markets Stripe does not, and for domestic revenue they are often the correct answer.

Table 2.1 Options when your jurisdiction is not directly supported. Only the first two give you a Stripe account, and both are legitimate structures rather than workarounds.

TRAP

There is a persistent genre of advice suggesting you register with a nominee address, a virtual mailbox and a borrowed identity in a supported country. Do not. This is identity misrepresentation to a regulated financial institution. The realistic outcome is not a warning; it is account termination with funds held, because the account was obtained under false representations. Every payout you were relying on becomes contested at the worst moment.

2.2 Prohibited and restricted businesses

Stripe publishes a list of business categories it will not serve, and a second list it will serve only under conditions. The full text is maintained on Stripe's legal pages and it is updated; treat the summary below as a map, not as the territory.

The prohibited list falls into recognisable clusters, and understanding the cluster is more useful than memorising the entries, because it lets you predict where a novel business model will land.

CLUSTER	REPRESENTATIVE CATEGORIES	WHY THE NETWORK REFUSES IT
Illegal or sanctioned	Anything unlawful in the relevant jurisdiction, sanctioned parties and territories, counterfeit goods, stolen property.	Non-negotiable legal exposure for the acquirer and for Stripe.
Regulated substances and devices	Cannabis and derivatives in many contexts, prescription drugs without proper licensing, tobacco and vaping products, weapons and ammunition, explosives.	Licensing regimes vary by state and country, and the network cannot verify compliance at transaction time.
Financial services adjacency	Money transmission, currency exchange, certain cryptocurrency activities, debt collection, credit repair, check cashing, wire transfer services.	These are separately licensed activities. Processing them through a payment facilitator would place Stripe in a regulatory role it has not registered for.
Gambling and speculative play	Real money gambling, lotteries, sweepstakes, sports betting, fantasy sports with prizes, and skill-based competitions with cash entry.	Heavily licensed and jurisdiction specific. Some of this is available with prior approval in specific countries.
High chargeback models	Negative option billing, free trials converting silently, get-rich-quick offers, pyramid and multi-level marketing schemes, unsolicited outbound telemarketing sales.	Network chargeback thresholds. Portfolios that exceed them attract fines and remediation programmes.

CLUSTER	REPRESENTATIVE CATEGORIES	WHY THE NETWORK REFUSES IT
Adult and exploitative content	Pornography and sexually oriented services, and, absolutely, any material involving minors.	Card network brand rules plus criminal exposure in the last case.
Deceptive or predatory	Multi-level marketing, essay mills, ticket scalping in some markets, aggregators re-selling others' work, and services designed to mislead.	Consumer protection rules and reputational risk to the brand.

Table 2.2 Clusters of restricted activity. The list is directional. The authoritative version is Stripe's published restricted businesses page, which should be read in full before onboarding an unusual model.

CONTEXT

Two subtleties catch honest businesses. First, categories can be permitted *with prior written approval*, which means the answer to "can I do this on Stripe" is sometimes "yes, after a conversation", not "no". If your model sits near a line, ask before you build. Second, the restriction applies to the *activity*, not the label. Describing a lending product as a subscription does not change what it is, and the risk team will classify it correctly eventually, usually after you have customers.

Restricted uses of Stripe's own products

Separately from the business categories, Stripe restricts certain *uses* of the platform regardless of what you sell. These are the ones engineers accidentally violate:

- Processing your own cards through your own account, or any transaction where you are on both sides. This is treated as cash advance and it terminates accounts.
- Aggregating payments on behalf of third parties without using Connect. If money passes through your account on its way to someone else, you are a facilitator, and there is a product for that with a compliance model attached.
- Using test data or the API in a way that constitutes probing, load generation against production, or automated account creation.
- Sharing a single account across genuinely distinct legal entities.

2.3 Verification, and why accounts get paused

Onboarding is a know-your-customer and know-your-business process. Stripe collects identifying information about the entity and about beneficial owners, and it verifies that information against external sources. The volume and timing of what it asks for depends on the country, the entity type and the risk profile of the category.

Typical requirements: legal entity name and registration number, business address, tax identifier, the identity of directors and of beneficial owners above a threshold, a bank account in the entity's name, a description of the product and, frequently, a live website that matches that description.

PRACTICE

Publish the boring pages before you request activation. A clear description of what you sell, prices in a visible currency, terms of service, a refund policy, a privacy policy and a reachable contact address. Reviews stall far more often on a site that does not visibly match the stated business than on anything technical. This costs an afternoon and removes the most common cause of onboarding delay.

Accounts already in production get restricted for a handful of reasons, and they are worth knowing in advance because the remediation is different in each case:

TRIGGER	WHAT IT LOOKS LIKE	WHAT TO DO
Unanswered verification request	Payouts pause; the dashboard shows an outstanding requirement.	Watch the <code>account.updated</code> event and alert a human. This one is entirely preventable.
Business does not match the description	Review opened after a manual look at your site or a customer complaint.	Update the account description to reflect what you actually sell now, not what you sold at signup.
Dispute rate above network thresholds	Warning, then reserve, then possible termination.	Chapter 13. This is a product and support problem before it is a payments problem.
Card testing traffic	Spike of low value declines, sometimes network fines.	Chapter 12. Rate limit the endpoint and enable the relevant Radar protections.
Sudden volume change	Risk review, possible reserve.	Tell Stripe before a launch that will multiply volume. A predicted spike is handled very differently from a surprise.

Table 2.3 Common causes of account restriction and their remediation. Only the first is purely administrative; the rest have engineering components.

Reserves

For businesses with delayed delivery, high refund rates or elevated risk categories, Stripe may hold a rolling reserve: a percentage of volume retained for a defined period before payout. This is standard practice across the acquiring industry, not a punishment, but it has real cash flow consequences. A business selling annual plans for services delivered over twelve months carries genuine forward liability, and the reserve reflects that. Model it before it appears.

RULE

Resolve eligibility, entity and jurisdiction before you design the integration. A perfectly architected money path is worth nothing if the account it runs through cannot be activated, and the architecture itself sometimes changes based on the answer, particularly if the outcome is a merchant of record or Connect.

CHAPTER THREE

Choosing an Integration Shape

How you collect the payment sets your compliance scope, your engineering cost, how well the checkout converts and how much of the experience you control. It is also the hardest decision to reverse later.

Stripe offers several ways to collect a payment, and they are not merely stylistic alternatives. They differ in one structural respect that determines everything else: **how close the raw card number comes to code you wrote**. That single variable drives your PCI questionnaire, the attack surface of your payment page, and the amount of regulatory machinery you have to build yourself.

3.1 The five shapes

Payment Links

A URL Stripe hosts. You create it in the dashboard or through the API, share it, and Stripe handles the entire transaction. No frontend code at all. Appropriate for invoices, one-off sales, creators and validating demand before building anything.

Stripe-hosted Checkout

Your server creates a Checkout Session and redirects the browser to a page on Stripe's domain. Stripe renders the form, collects the card, handles authentication, applies tax if configured, and redirects back. The card never touches your origin. This is the default recommendation for the large majority of businesses.

Embedded Checkout

The same Checkout Session, rendered inside an iframe on your own page. The customer never leaves your domain visually, but the card fields still live in a document served by Stripe, so the card data still does not enter your origin. You get more layout control at the cost of a little more frontend work.

Elements and the Payment Element

Individual UI components, hosted in Stripe-controlled iframes, that you compose into a form you design. The Payment Element in particular renders whichever payment methods are relevant to the customer's location and your configuration, and it handles authentication flows. Maximum design control while keeping card fields out of your document. This is the right answer when checkout is genuinely part of your product experience.

Direct API integration

Your own form collects the card number and your server sends it to Stripe. The primary account number transits your infrastructure. Almost nobody should do this. It exists for specific cases: certain in-person and Terminal flows, and organisations that already operate a fully certified cardholder data environment for other reasons.

3.2 The compliance consequence

The choice above maps almost directly onto which PCI self assessment questionnaire you are eligible for, and the difference between the lightest and heaviest is roughly a factor of ten in the number of controls you must attest to and evidence.

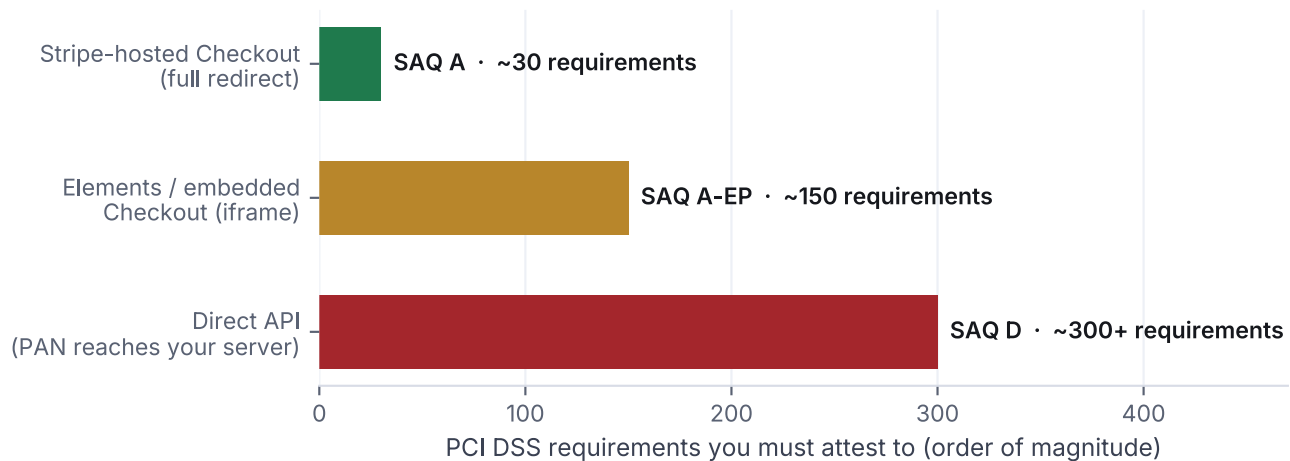


Figure 3.1 Approximate PCI DSS scope by integration shape. Figures are orders of magnitude for the relevant self assessment questionnaire, not exact counts; confirm the applicable SAQ with your acquirer or QSA. Handling card data directly can put more than three hundred controls in scope.

RULE

Choose the shape that keeps the primary account number furthest from your code, and only move closer when a concrete product requirement forces it. "We want the checkout to look like our brand" is satisfied by the Payment Element. It is not a reason to touch card numbers.

3.3 Full comparison

DIMENSION	PAYMENT LINKS	HOSTED CHECKOUT	EMBEDDED CHECKOUT	PAYMENT ELEMENT	DIRECT API
PCI questionnaire	SAQ A	SAQ A	SAQ A	SAQ A	SAQ D
Card data touches your origin	No	No	No, iframe is Stripe's	No, iframes are Stripe's	Yes
Customer leaves your domain	Yes	Yes	No	No	No
Frontend engineering	None	Minimal	Low	Moderate	High
Design control	Logo and colours	Logo and colours	Surrounding page	Near total	Total

DIMENSION	PAYMENT LINKS	HOSTED CHECKOUT	EMBEDDED CHECKOUT	PAYMENT ELEMENT	DIRECT API
Payment methods	Configured centrally	Configured centrally, adapts to customer	Same	Same, dynamic	You build each one
SCA and 3DS handling	Automatic	Automatic	Automatic	Automatic	You orchestrate
Address, tax, promotion codes	Built in	Built in	Built in	You compose from other Elements	You build
Subscription support	Yes	Yes	Yes	Yes, with more wiring	Yes, with much more
Time to first live payment	Minutes	Hours	A day	Days	Weeks to months
Ongoing maintenance burden	Negligible	Low	Low	Moderate	Continuous

Table 3.1 The shapes compared. Note that SAQ A eligibility is common to the first four, so the compliance argument for direct API integration essentially never holds for a normal commerce business.

3.4 A decision procedure

Rather than weighing all eleven dimensions, answer four questions in order. The first answer that applies is your answer.

STEP	QUESTION	IF YES
1	Do you need to collect payment without building or deploying anything, for a small number of items or invoices?	Payment Links. Stop here. Revisit when volume or product complexity justifies more.
2	Is checkout a step users pass through rather than a place they spend time, and are you content with Stripe's layout?	Stripe-hosted Checkout. This covers most SaaS, most digital goods and a large share of ecommerce.
3	Is the redirect itself the objection, for conversion or brand reasons, but Stripe's form layout is otherwise acceptable?	Embedded Checkout. Same session object, same compliance posture, no navigation away.
4	Is the checkout experience genuinely part of your product: multi-step, conditional, integrated with your own flows?	Payment Element, composed with the other Elements you need.
5	None of the above, and you already operate an audited cardholder data environment.	Direct API. Confirm with your QSA first, and read Chapter 4 twice.

Table 3.2 The decision ladder. In several hundred integrations, step 5 is the correct destination for a vanishingly small number.

PRACTICE

Start at step 2 even if you expect to end at step 4. Hosted Checkout gets you a working, compliant money path in an afternoon, which means the rest of the system, webhooks, fulfilment, reconciliation, gets built and tested against real events early. Migrating to the Payment Element later touches only the frontend and the session creation call. The server side you built stays. Starting at step 4 means debugging the money path and the UI simultaneously.

3.5 The one-way doors

Most of this decision is reversible. Three parts of it are not, or are painful enough to treat as one-way, and they deserve deliberate thought now rather than discovery later.

Touching card data

Once the primary account number has entered your infrastructure, even once, even in a log line, your scope has expanded and unwinding it requires proving that no residual data remains. The cheapest moment to decide never to touch card data is before you have.

Becoming a platform

Paying out to third parties, splitting a payment between yourself and a seller, or holding funds on someone's behalf are Connect activities with their own onboarding, liability model and compliance obligations. Retrofitting Connect onto a direct-charge codebase is a rewrite of the money path, not a feature. Chapter 14.

Your customer and subscription data model

Whether you treat Stripe as the source of truth for entitlement or maintain your own mirror determines the shape of every webhook handler you will write. Chapter 10 argues for a specific answer. Changing it later means a backfill against live billing state, which is exactly as pleasant as it sounds.

Everything else in this book assumes you chose a shape where card data never reaches your servers. If that is not true for you, the security burden multiplies, and this book becomes a prerequisite rather than a complete guide.

PART TWO

The Regulatory Surface

Three bodies of rules apply whether you have read them or not: the card networks' security standard, European authentication law, and the browser-side integrity requirements that the last decade of card skimming forced into both.

4 PCI DSS 4.0.1 in Practice

5 Strong Customer Authentication and 3D Secure

6 The Payment Page: TLS, CSP and E-Skimming

CHAPTER FOUR

PCI DSS 4.0.1 in Practice

The standard is long and the part that applies to a well designed integration is short. This chapter is about knowing which part that is, and what changed when the fourth revision made client side integrity mandatory.

The Payment Card Industry Data Security Standard is not law. It is a contractual regime: the card networks require it of acquirers, acquirers require it of merchants, and the consequences of non-compliance are commercial rather than criminal. Fines, higher rates, liability shifts after a breach, and in serious cases losing the ability to accept cards at all. It applies to every entity that stores, processes or transmits cardholder data, which includes you even when Stripe does the heavy lifting.

4.1 Levels and questionnaires

Your obligations scale with volume. Merchant levels are defined per card network and by annual transaction count, and the practical distinction is between merchants who self assess and merchants who are audited by a Qualified Security Assessor.

LEVEL	ROUGHLY	VALIDATION
1	Above six million card transactions annually, or any merchant that has suffered a breach.	Annual on-site assessment by a QSA, quarterly network scans by an Approved Scanning Vendor.
2	One to six million transactions annually.	Annual self assessment questionnaire, quarterly scans.
3	Twenty thousand to one million ecommerce transactions.	Annual SAQ, quarterly scans.
4	Below twenty thousand ecommerce transactions.	Annual SAQ, scans as required by the acquirer.

Table 4.1 Merchant levels. Thresholds differ slightly between networks; your acquirer tells you definitively which level you are. Stripe itself is a Level 1 service provider, which is a different and stricter category.

The questionnaire you complete depends on how you accept payments, which is exactly the decision you made in Chapter 3.

SAQ	WHO IT APPLIES TO	PRACTICAL WEIGHT
A	Ecommerce merchants who fully outsource cardholder data functions to a validated third party. Hosted Checkout, embedded Checkout, Payment Links and Elements all land here.	Lightest. Around thirty requirements, and since the 4.0 revision it explicitly includes obligations about the scripts on your payment page.

SAQ	WHO IT APPLIES TO	PRACTICAL WEIGHT
A-EP	Merchants whose own site does not receive card data, but where the site controls how the payment page is delivered or redirects to the processor in a way that could affect the transaction.	Substantially heavier. Roughly five times the requirement count of SAQ A, including vulnerability management and secure development obligations.
D	Any merchant that stores, processes or transmits cardholder data on its own systems.	The full standard. Segmentation, key management, logging, penetration testing, formal secure development lifecycle. Hundreds of controls.

Table 4.2 The questionnaires. Confirm eligibility with your acquirer; several conditions attach to SAQ A that a careless integration can break.

PRACTICE

Stripe's dashboard reports the questionnaire your integration appears to qualify for, and for many small merchants generates the attestation directly. Use it as a starting point and not as a conclusion, because it reflects the integration Stripe can observe. If you have a custom flow, an unusual redirect, or any path that could handle card data, verify with your acquirer.

4.2 The twelve requirements, and where each one lands

#	REQUIREMENT	WHO CARRIES IT IN A HOSTED INTEGRATION
1	Install and maintain network security controls	Stripe for the vault. You for your own network, minimally in SAQ A.
2	Apply secure configurations to all system components	You, for anything you operate. No vendor defaults, no shared accounts.
3	Protect stored account data	Stripe , provided you store none. This is the single largest reason to stay hosted.
4	Protect cardholder data with strong cryptography during transmission	Shared. Stripe enforces TLS on its side; you must serve your own pages over HTTPS.
5	Protect systems and networks from malicious software	You, scoped to your own environment.
6	Develop and maintain secure systems and software	You . Includes 6.4.3, the payment page script inventory. See below.
7	Restrict access to system components by business need to know	You. Directly maps to Stripe dashboard roles and restricted API keys.
8	Identify users and authenticate access	You. Unique accounts, multi-factor authentication on the dashboard, no shared logins.
9	Restrict physical access to cardholder data	Stripe, in a hosted integration.
10	Log and monitor all access to system components and cardholder data	You for your systems. Stripe's API logs and audit trail help.

#	REQUIREMENT	WHO CARRIES IT IN A HOSTED INTEGRATION
11	Test security of systems and networks regularly	You. Includes 11.6.1, payment page tamper detection. See below.
12	Support information security with organisational policies and programmes	You. The paperwork requirement people forget until the questionnaire.

Table 4.3 The twelve requirement families of PCI DSS 4.0.1 and where responsibility sits when card data never touches your servers.

4.3 What you may keep and what you may never keep

This distinction predates every version of the standard and it has never softened.

DATA ELEMENT	STORAGE	NOTES
Primary account number (PAN)	Permitted, rendered unreadable	If you store it you are in SAQ D. In practice: never store it. Store Stripe's identifiers instead.
Cardholder name	Permitted	Protect it as personal data under your own privacy obligations.
Expiry date	Permitted	Only alongside a permitted PAN representation.
Service code	Permitted	Rarely relevant to an ecommerce merchant.
Last four digits and card brand	Permitted	Stripe returns these on the payment method object. Use them for receipts and support.
Card verification value (CVV, CVC, CID)	Never, after authorisation	Not in a database, not in a log, not in an error report, not for five minutes.
Full magnetic stripe or chip track data	Never	Applies to in-person integrations.
PIN or PIN block	Never	No exceptions of any kind.

Table 4.4 Sensitive authentication data may not be retained after authorisation, even encrypted. This is the bright line of the standard.

TRAP

The most common way merchants store prohibited data is not a database column. It is a log line, an error tracker payload, a support ticket screenshot, or a request body captured by an application performance monitoring agent. Anywhere you serialise a request wholesale, you have built an accidental card data store. Scrub at the source, not at the destination, and explicitly deny-list payment fields in your observability pipeline.

4.4 What changed in version 4, and why it matters to you

Version 4.0 of the standard was written in response to a decade of client-side skimming attacks. In those attacks nothing on the merchant's server is compromised. A script loaded by the payment page is modified, and it reads the card fields as the customer types. The merchant's infrastructure is clean, the TLS is valid, the server logs show nothing, and cards are exfiltrating continuously.

The standard's answer was two new requirements. Both were classified as best practice during the transition period and became **mandatory on 31 March 2025**. Crucially, they apply to SAQ A merchants, which means they apply to you even with a fully hosted integration.

REQUIREMENT	WHAT IT DEMANDS	WHAT IT MEANS OPERATIONALLY
6.4.3	Manage all payment page scripts: authorise each one, justify its necessity, and maintain an inventory.	You need a written list of every script the payment page loads, including transitive ones, with an owner and a reason for each. Tag managers and analytics tools that inject scripts dynamically make this genuinely difficult.
11.6.1	Deploy a change and tamper detection mechanism that alerts on unauthorised modification of the HTTP headers and content of the payment page.	You need monitoring that notices when the payment page changes unexpectedly, at least weekly. Subresource integrity, CSP reporting, and dedicated client-side monitoring tools all contribute; none alone is automatically sufficient.

Table 4.5 The two client-side requirements introduced in PCI DSS 4.0 and mandatory since March 2025. They are the reason Chapter 6 exists.

RULE

Using Stripe-hosted Checkout does not exempt you from 6.4.3 and 11.6.1. The page that redirects to Stripe, and any page where an Element iframe is embedded, is a payment page under the standard. Inventory its scripts and monitor it for change.

4.5 Reducing scope deliberately

Scope reduction is the highest return activity in payment compliance, and it is mostly architectural rather than procedural.

- **Never let the PAN into your process.** Hosted fields, iframes, redirects. Everything else follows from this.
- **Store identifiers, not instruments.** A `cus_`, a `pm_`, a `sub_` and a last four are enough to run a business, and none of them is cardholder data.
- **Keep the payment page thin.** Every script on it is an inventory entry and a monitoring obligation under 6.4.3 and 11.6.1. A checkout page with no analytics, no chat widget and no tag manager is both safer and cheaper to attest.
- **Segment.** If any part of your estate does touch card data, isolate it at the network level so that the rest of the estate falls outside scope.
- **Use restricted API keys.** Requirement 7 is about least privilege, and restricted keys are the direct implementation of it. Chapter 7.

SAQ A What a hosted integration should qualify for	Zero Card numbers that should exist anywhere in your systems	Annual Cadence for re-attestation and for reviewing the script inventory
--	--	--



CHAPTER FIVE

Strong Customer Authentication and 3D Secure

European rules require two independent factors on most online card payments. This chapter covers how Stripe requests them for you, and the mistakes that quietly cost conversion and produce declines nobody can explain.

The second Payment Services Directive introduced Strong Customer Authentication for electronic payments in the European Economic Area, with an equivalent regime retained in the United Kingdom. The requirement is that the customer be authenticated using at least two of three independent factors:

FACTOR	DEFINITION	IN PRACTICE
Knowledge	Something only the customer knows	A password or a passcode set with the issuing bank. Not the card number, which is not secret.
Possession	Something only the customer has	A registered phone receiving a push from the banking app, a hardware token, a card reader.
Inherence	Something the customer is	Fingerprint or face recognition, typically performed inside the banking app.

Table 5.1 The three authentication factors. Two must be from different categories, and compromise of one must not compromise the other.

The mechanism that delivers this for card payments is 3D Secure, in its second generation. The name describes three domains: the issuer, the acquirer and the interoperability layer between them. Version 2 improved substantially on version 1 by passing far richer contextual data to the issuer, which allows most transactions to be authenticated *frictionlessly*, with no challenge shown to the customer at all.

5.1 Frictionless, challenge, and the liability shift

When a payment is submitted for authentication, the issuer receives dozens of data points: device fingerprint, transaction history, amount, merchant, shipping address consistency. It then decides between two outcomes. In a **frictionless** flow it approves authentication silently and the customer notices nothing. In a **challenge** flow it asks for a factor, typically by pushing a prompt to the banking app.

The commercial significance is the liability shift. When a transaction is successfully authenticated with 3D Secure, liability for a subsequent fraudulent-transaction chargeback generally moves from the merchant to the issuer. You lose the dispute far less often, because the issuer authenticated the customer, not you.

CONTEXT

This creates a genuine tension. Authentication improves your fraud position and is legally required in the EEA and UK, but every challenge shown is an opportunity for a customer to abandon. The exemption framework exists precisely to manage that tension, and the whole point of using Checkout or the Payment Element is that Stripe navigates it for you, requesting authentication when required and applying exemptions when they are available.

5.2 The exemptions

Not every transaction requires a challenge. The regulation defines exemptions, and the issuer decides whether to honour the one you request. That last clause matters: an exemption is a request, never a guarantee.

EXEMPTION	CONDITION	NOTES
Low value	Transaction below EUR 30.	Cumulative counters apply: the issuer must challenge after a run of consecutive exempt payments or once a cumulative amount is reached. You cannot rely on it indefinitely.
Transaction risk analysis	The acquirer's fraud rate is below a threshold, permitting exemption up to a corresponding value ceiling.	Three tiers, shown in Figure 5.1. Depends on the acquirer's portfolio fraud rate, not yours alone.
Recurring, fixed amount	Same amount, same payee, on a schedule.	The first payment must be authenticated. Subsequent ones inherit. Any change to the amount restarts the requirement.
Merchant initiated transactions	Charged without the customer present, under a mandate agreed earlier.	Out of scope of SCA rather than exempt. Requires a properly set up saved payment method. See 5.3.
Trusted beneficiary	The customer has added you to a list of trusted merchants at their bank.	Whitelisting is done during a previous authenticated payment, if the issuer offers it.
Corporate cards	Secure corporate payment processes, lodged cards, virtual card numbers in a managed travel context.	Narrow, and it depends on the card product.
Mail order and telephone order	Card details taken over the phone or by post.	Out of scope entirely, but MOTO carries its own fraud profile and its own dispute risk.
One-leg-out	The issuer or the acquirer is outside the EEA and UK.	Best efforts basis. This is why a US merchant selling to EU customers still sees challenges.

Table 5.2 SCA exemptions and out-of-scope cases. Stripe requests the applicable exemption automatically in Checkout and the Payment Element.

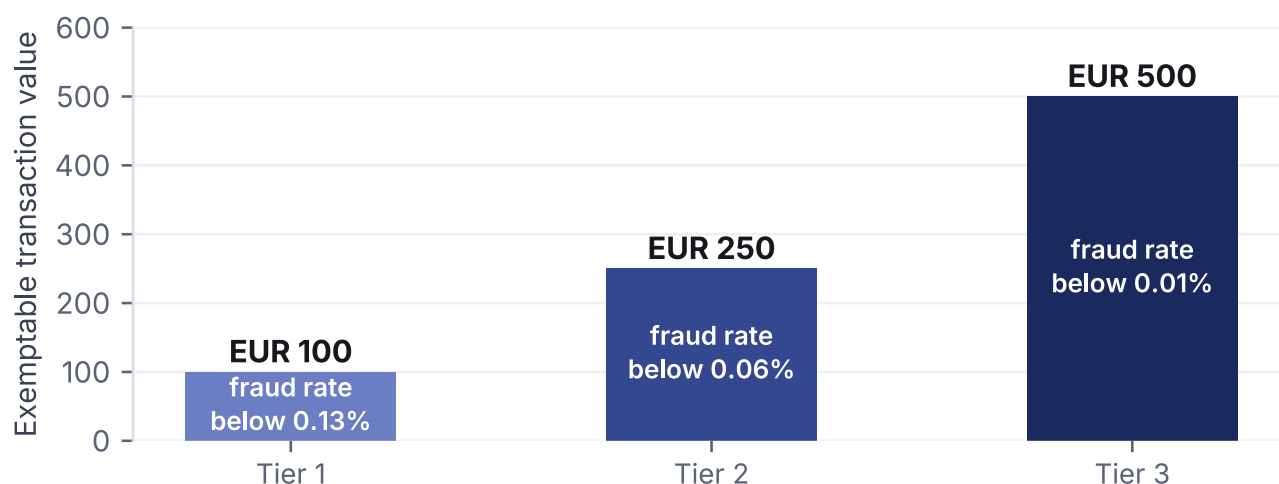


Figure 5.1 Transaction risk analysis tiers. The lower the acquirer's fraud rate, the higher the transaction value that may be exempted from a challenge. Thresholds are set by regulation and should be verified against the current text before being relied on commercially.

5.3 Saving cards for later, correctly

The single most common SCA implementation bug is charging a saved card off-session without having set it up properly on-session. The payment is refused, the customer is not present to authenticate, and the subscription silently fails to renew.

The rule is that authentication and consent must be captured while the customer is present, and the resulting payment method must be recorded as being for future off-session use.

```
// On-session: the customer is in front of you.
// Declare the future use at creation time.
const session = await stripe.checkout.sessions.create({
  mode: "subscription",
  line_items: [{ price: TRUSTED_PRICE_ID, quantity: 1 }],
  customer: customerId,
  // For one-off payments where you intend to charge again later:
  // payment_intent_data: { setup_future_usage: "off_session" },
  success_url: `${BASE}/return?session_id={CHECKOUT_SESSION_ID}`,
  cancel_url: `${BASE}/pricing`,
});
```

Declaring intent up front lets Stripe request the correct authentication and record the mandate, so later charges are in scope of the recurring or MIT exemption.

```
// Off-session, later: charge the saved method.
try {
  await stripe.paymentIntents.create({
    amount: computedAmountInMinorUnits, // from your own pricing data
    currency: "eur",
    customer: customerId,
    payment_method: savedPaymentMethodId,
    off_session: true,
    confirm: true,
  }, { idempotencyKey: `renewal:${subscriptionId}:${periodStart}` });
} catch (err) {
  if (err.code === "authentication_required") {
    // Expected. Do not retry blindly. Email the customer a link
    // to complete authentication on-session, then charge again.
    await requestOnSessionAuthentication(customerId, err.raw.payment_intent.id);
  } else {
    throw err;
  }
}
```

The `authentication_required` error is not a failure to handle generically. It is a specific instruction to bring the customer back on-session.

TRAP

Retrying an `authentication_required` decline on a timer will never succeed, because the condition is structural and not transient. Teams add it to a generic retry queue, watch renewals fail for weeks, and conclude that European cards are unreliable. They are not. The integration is asking the wrong question repeatedly.

5.4 What you actually have to build

If you use Checkout or the Payment Element, the answer is close to nothing, and that is the argument for them restated in regulatory terms. Stripe determines whether authentication is required, requests the appropriate exemption, renders the challenge and surfaces the result.

What remains yours:

- Declare future usage at the moment of collection, as above.
- Handle `requires_action` correctly if you are driving the confirmation flow yourself rather than letting Checkout do it.
- Handle `authentication_required` on off-session charges by bringing the customer back, not by retrying.
- Subscribe to `invoice.payment_action_required` and act on it, because that is how a subscription tells you a renewal is stuck behind a challenge.
- Never assume a payment succeeded because the browser reached your success page. Chapter 10.

RULE

Authentication state is a server-side fact carried on the `PaymentIntent`, not a client-side event. The only reliable statement about whether a payment is complete comes from the `PaymentIntent` status or from a verified webhook, and never from the fact that a redirect happened.

PaymentIntent lifecycle

Only succeeded, and for delayed methods the asynchronous success event, should ever unlock value.

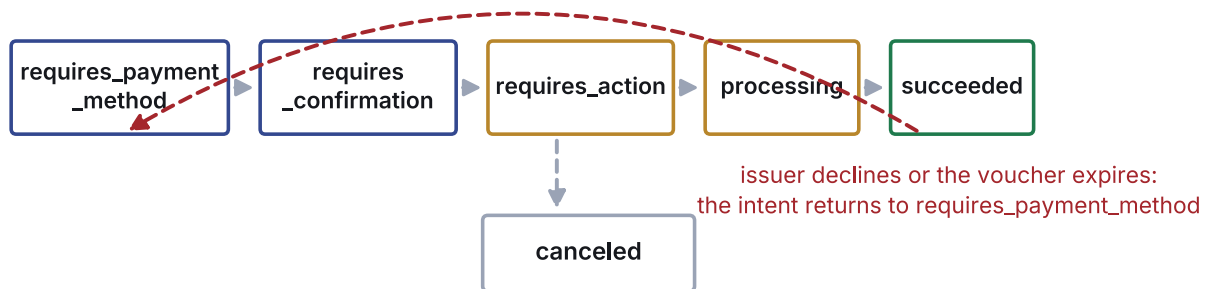


Figure 5.2 The `PaymentIntent` lifecycle. Authentication lives in `requires_action`. Note that a failed authentication returns the intent to `requires_payment_method` rather than terminating it, which is why a single intent can carry several attempts.

CHAPTER SIX

The Payment Page: TLS, CSP and E-Skimming

This attack never reaches your server. A modified script on a page you control reads the card fields as the customer types, and your logs stay clean.

Between roughly 2015 and today, a family of attacks collectively known as e-skimming, or Magecart after one of the groups practising it, has produced more compromised card numbers than almost any other technique against ecommerce. The method is elegant and depressingly simple. Rather than breaking into the merchant's database, the attacker modifies a JavaScript file the checkout page loads. The script reads input fields and posts their values to a server the attacker controls. Everything continues to work. The customer completes the purchase. The merchant sees nothing wrong for months.

The compromised script is rarely the merchant's own. It is typically a third party: an analytics library, a live chat widget, a personalisation service, an A/B testing tool, a transitively pulled npm dependency, or a script injected by a tag manager that a marketing team controls without engineering review.

Your payment page is only as trustworthy as the least trustworthy script it loads, and you probably do not know what that is.

6.1 Why iframes help, and where they stop helping

Stripe's hosted fields work because of the same-origin policy. When the card number is typed into an input inside an iframe served from Stripe's domain, JavaScript running on your page cannot read it. A skimmer injected into your page has no access to the field's value, cannot attach a listener to it, and cannot serialise it.

This is a strong protection and it is the primary reason Elements and Checkout reduce your PCI scope. But it has boundaries that are worth stating precisely, because teams sometimes conclude that iframes make the payment page irrelevant.

WHAT THE IFRAME STOPS	WHAT IT DOES NOT STOP
Reading the card number, CVC or expiry from the DOM.	Removing the legitimate iframe and replacing it with a convincing look-alike form on your own origin.
Keylogging the card fields directly.	Overlaying a transparent input on top of the iframe.
Serialising the payment form on submit.	Reading the customer's name, email, billing address and order contents, all of which sit on your page.
Intercepting the tokenisation request.	Rewriting your success handler so the order is redirected, or the amount displayed differs from the amount charged.

WHAT THE IFRAME STOPS	WHAT IT DOES NOT STOP
Direct exfiltration of the PAN.	Redirecting the entire checkout to an attacker-controlled clone before the iframe ever loads.

Table 6.1 The limits of iframe isolation. It removes the easiest attack and leaves several harder ones, which is precisely why PCI DSS 4.0 added script management and tamper detection rather than accepting hosted fields as sufficient.

6.2 Transport, and the headers that matter

Start with the non-negotiable baseline. Every page in the payment flow, not just the one with the card fields, is served over HTTPS with a valid certificate, and downgrade is impossible.

HEADER	VALUE	PURPOSE
<code>Strict-Transport-Security</code>	<code>max-age=63072000; includeSubDomains; preload</code>	Removes the plaintext downgrade window entirely. Preload only once you are certain every subdomain can serve HTTPS.
<code>Content-Security-Policy</code>	See Appendix B	The core control for 6.4.3. Restricts which origins can supply scripts, frames and network destinations.
<code>X-Content-Type-Options</code>	<code>nosniff</code>	Stops the browser reinterpreting a response as script.
<code>Referrer-Policy</code>	<code>strict-origin-when-cross-origin</code>	Prevents session identifiers and order references leaking in referrer headers to third parties.
<code>Permissions-Policy</code>	Deny what you do not use	Camera, microphone, geolocation. Payment pages need almost none of it.
<code>Cross-Origin-Opener-Policy</code>	<code>same-origin-allow-popups</code>	Isolates your browsing context while still permitting the popups some payment methods use.
<code>Cache-Control</code>	<code>no-store</code> on authenticated pages	Keeps order and customer data out of shared caches and browser history.

Table 6.2 Response headers for pages in the payment flow. Note the deliberate absence of `X-Frame-Options`: use the CSP `frame-ancestors` directive instead, which supersedes it.

6.3 Content Security Policy as a compliance control

A Content Security Policy is the mechanism by which requirement 6.4.3 becomes enforceable rather than aspirational. An inventory of authorised scripts is a document. A CSP is that document expressed in a form the browser enforces, and any attempt to load a script outside it becomes a violation report rather than an exfiltration.

Stripe's client libraries require specific origins to be permitted. The full directive set by product is in Appendix B; the shape of a working policy for a Checkout or Elements page is:

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' https://js.stripe.com;
  frame-src https://js.stripe.com https://hooks.stripe.com https://checkout.stripe.com;
  connect-src 'self' https://api.stripe.com https://maps.googleapis.com;
  img-src 'self' data: https://*.stripe.com;
  style-src 'self' 'unsafe-inline';
  frame-ancestors 'none';
  base-uri 'self';
  form-action 'self';
  object-src 'none';
  report-uri /csp-report
```

A starting policy. Verify the current required origins against Stripe's documentation, since they change as products are added, and tighten `style-src` once you have removed inline styles.

PRACTICE

Deploy in report-only mode first, using `Content-Security-Policy-Report-Only`, and collect violations for a full business cycle before enforcing. You will discover scripts nobody remembers adding, which is itself the 6.4.3 inventory exercise. Once enforcing, keep the report endpoint: violations in production are your tamper detection signal under 11.6.1.

Why `unsafe-inline` for scripts is not acceptable here

A policy containing `script-src 'unsafe-inline'` provides almost no protection, because the most common injection vector is an inline script. If you cannot remove inline scripts immediately, use nonces or hashes. On a checkout page specifically, the effort is justified: this is the one page where the policy has to actually hold.

```
// Generate a fresh nonce per response, never reuse across requests.
const nonce = crypto.randomBytes(16).toString("base64");
res.setHeader("Content-Security-Policy",
  `script-src 'nonce-${nonce}' https://js.stripe.com; object-src 'none'; base-uri 'self'`);
// then render: <script nonce="${nonce}">...</script>
```

6.4 Subresource integrity, and its one hard limit

Subresource integrity pins a cryptographic hash to a script tag, so the browser refuses to execute a file that has changed. It is the right tool for pinned, versioned third party libraries.

```
<script src="https://cdn.example.com/widget-4.2.1.js"
  integrity="sha384-oqVuAfXRKap7fdgcCY5uykM6+R9GqQ8K/uxy9rx7HNQlGyL1kPzQho1wx4JwY8wC"
  crossorigin="anonymous"></script>
```

TRAP

Do not apply SRI to <https://js.stripe.com/v3/> or to Stripe's other loader scripts. Stripe updates them continuously and deliberately, which is how security fixes and new payment methods reach your page without you deploying. A pinned hash will break your checkout the next time they ship. The correct control for Stripe's script is the CSP allow-list, not integrity pinning. The same reasoning applies to any vendor that publishes an unversioned evergreen URL.

6.5 Tamper detection under 11.6.1

Requirement 11.6.1 asks for a mechanism that alerts on unauthorised modification of the payment page's headers and content, evaluated at least weekly. There is no prescribed product. A defensible implementation typically combines several of these:

CSP violation reporting

Collect reports and alert on any violation from a payment page. In a properly tuned policy, a violation from checkout is either a deployment mistake or an attack, and both need a human.

Synthetic monitoring with content hashing

A scheduled job loads the payment page in a headless browser, enumerates every script and resource actually fetched, hashes the response bodies, and compares against a known-good manifest. This catches dynamically injected scripts that a static scan of your repository misses. Weekly satisfies the standard; hourly is not expensive.

Header snapshotting

The requirement covers headers explicitly. Assert the presence and value of the security headers in Table 6.2 from an external vantage point, because a CDN or edge configuration change can strip them without any deploy in your application repository.

Dependency and build integrity

Lockfiles committed and enforced in CI, dependency review on updates, and a build that fails on unexpected additions to the bundle for the checkout route. Bundle size assertions catch a surprising amount.

Dedicated client-side security tooling

Commercial products exist specifically to satisfy 6.4.3 and 11.6.1. They are worth considering when the payment page necessarily carries many third party scripts, which is common in retail and rare in SaaS.

6.6 Reducing the problem instead of monitoring it

Every control above is a mitigation for scripts you have chosen to load. The cheapest strategy is to load fewer.

RULE

The checkout route carries no tag manager, no session replay, no chat widget, no A/B testing framework and no advertising pixel. Measure conversion by observing server-side events, which are more accurate anyway. Every script you remove from that page is one fewer inventory entry, one fewer monitoring target and one fewer path to your customers' cards.

This is usually an organisational negotiation rather than a technical one. The argument that works is the compliance one: each additional script on the payment page is a documented, justified, owned and monitored artefact under PCI DSS 4.0.1, with a named person responsible. Marketing teams reliably discover that they did not need the pixel on that specific page.

A short checklist for the page itself

- ☐ Served over HTTPS with HSTS, and every asset it loads is also HTTPS.
- ☐ CSP enforced, no `unsafe-inline` in `script-src`, reports collected.
- ☐ `frame-ancestors 'none'` unless you genuinely need to be framed.
- ☐ Complete written inventory of every script, with owner and justification.
- ☐ No analytics, chat, replay or tag manager on the route.
- ☐ Automated content and header comparison against a known-good manifest, at least weekly.
- ☐ SRI on pinned third party scripts, and deliberately not on Stripe's loader.
- ☐ Lockfiles enforced in CI, with review on dependency changes touching the checkout bundle.

PART THREE

The Server Side

The engineering core. Secrets and their blast radius, the payment flow that only the server may decide, the webhook endpoint everything else depends on, fulfilment that survives being run twice, and behaviour under failure.

- 7 Keys, Secrets and Blast Radius
- 8 The Money Path: Server-Authoritative Payments
- 9 Webhooks: The Load-Bearing Wall
- 10 Fulfilment and Data Consistency
- 11 Rate Limits, Errors and Resilience

CHAPTER SEVEN

Keys, Secrets and Blast Radius

A secret key is not a password. Anyone holding it can move your money, and the platform cannot tell that person apart from you.

Stripe authenticates API requests with a single string. There is no second factor, no origin check, no device binding. Possession is authorisation. This makes the API pleasant to work with and makes key handling the most consequential operational discipline in the integration.

7.1 The credential inventory

CREDENTIAL	PREFIX	EXPOSURE	WHAT IT CAN DO
Publishable key	<code>pk_live_</code> <code>pk_test_</code>	Public	Identifies your account to Stripe.js. Can tokenise card details and confirm intents the client already holds a secret for. Designed to be visible in page source. Leaking it is not an incident.
Secret key	<code>sk_live_</code> <code>sk_test_</code>	Server only	Everything. Read every customer, every charge, every payout. Create charges and refunds. Modify subscriptions. Change bank details in some configurations. Total account authority.
Restricted key	<code>rk_live_</code> <code>rk_test_</code>	Server only	Exactly the resources and permissions you granted it, per resource, as read or write. The correct default for anything that is not your primary application.
Webhook signing secret	<code>whsec_</code>	Server only	Verifies that an inbound event genuinely came from Stripe. One per endpoint. Leaking it lets an attacker forge events you will believe. Chapter 9.
Client secret	<code>pi_..._secret_...</code> <code>seti_..._secret_...</code>	Client, scoped	Confirms one specific <code>PaymentIntent</code> or <code>SetupIntent</code> . Scoped to a single object, but it can complete or read that object, so it must reach only the customer it belongs to.
Connect account ID	<code>acct_</code>	Semi-public	Not a credential, but combined with a platform secret key it addresses a connected account. Treat as sensitive metadata.

Table 7.1 Stripe credentials. Only the publishable key and a correctly scoped client secret ever belong in a browser.

RULE

If a string begins with `sk_`, `rk_` or `whsec_`, it must never appear in client-side code, in a bundle, in a mobile binary, in a repository, in a CI log, in a screenshot, in a support ticket, or in a prompt to a third party service.

7.2 What a leaked live secret key actually costs

Teams underestimate this because the obvious attack, creating fraudulent charges, is not the worst one. Ranked by how much damage they do before anyone notices:

ACTION	DETECTION	CONSEQUENCE
Bulk export of the customer list	Very low	Names, emails, addresses, purchase history, card brands and last four for your entire customer base. A data breach with notification obligations, and no unusual money movement to trigger an alert.
Creating refunds to attacker-controlled destinations	Low	Direct loss. Refunds look routine in a dashboard and reconcile against real charges.
Registering an additional webhook endpoint	Very low	Persistent access to your entire event stream, surviving rotation of the key that created it. Almost nobody audits the endpoint list.
Modifying subscription prices or applying coupons	Low	Slow revenue leakage that reconciles perfectly, because the invoices are genuinely correct.
Creating charges	High	Loud, and mostly self-defeating for the attacker since payouts go to your bank account.

Table 7.2 Consequences of a leaked `sk_live_`, ordered by how long the damage continues before discovery. The quiet ones are the expensive ones.

7.3 Restricted keys and least privilege

Most code that talks to Stripe does not need to be able to do everything. A reporting job needs to read charges. A support tool needs to read customers and create refunds. A webhook processor needs to read a handful of objects. None of them needs the ability to modify payout bank accounts.

Restricted keys let you grant permissions per resource, as none, read, or write. They are the direct implementation of PCI requirement 7 and they collapse the blast radius of a leak from "the entire account" to "one job's specific scope".

COMPONENT	SUGGESTED SCOPE	RATIONALE
Main application backend	Write on the objects it creates: Checkout Sessions, Customers, PaymentIntents, Subscriptions. Read on Prices and Products.	The only component that genuinely needs broad write access, and even here Prices should be read-only.
Webhook processor	Read on the object types it re-fetches. Write only where it must reconcile.	It reacts to events; it should rarely originate state changes.
Reporting and analytics	Read only, on charges, balance transactions, payouts.	Often the widest-deployed key and the one most likely to leak into a data warehouse config.
Support tooling	Read on customers and charges, write on refunds only.	Frequently the most human-exposed surface in the company.

COMPONENT	SUGGESTED SCOPE	RATIONALE
Local development	Test mode keys exclusively.	There is no legitimate reason for a live key on a laptop.

Table 7.3 A workable key partition. The goal is that no single leaked credential grants both bulk read of customer data and write access to money movement.

7.4 Storage and handling

Where secrets live

- A dedicated secret manager: AWS Secrets Manager, GCP Secret Manager, HashiCorp Vault, Doppler, or your platform's encrypted environment variable store.
- Injected into the process at runtime as environment variables, never read from a file that ships with the application.
- Different values per environment, with live keys present only in production, and access to the production secret restricted to a small named group.

Where they must not live

- Any file tracked by version control, including `.env`, config templates, test fixtures, notebooks and infrastructure code.
- Any variable name prefixed `NEXT_PUBLIC_`, `VITE_`, `REACT_APP_`, `PUBLIC_` or `EXPO_PUBLIC_`. These are inlined into the client bundle by the build tool, silently and by design.
- CI logs. Echoing an environment variable during debugging writes it to a log that is often retained and sometimes public.
- Error tracking payloads, which frequently capture full environment or full request context.
- Container image layers. A secret added in an early layer and deleted in a later one is still in the image.

TRAP

The framework-prefix mistake is the most common way a live secret key ends up in a browser bundle, and it produces no error at all. The application works perfectly. The key is simply also published to every visitor. Add a build-time assertion that fails the build if any environment variable matching the public prefix contains a value starting with `sk_`, `rk_` or `whsec_`. It takes ten lines and it has caught this in more than one codebase I have reviewed.

```
// build-guard.ts, executed before the bundler runs
const FORBIDDEN = /^(sk_|rk_|whsec_)/;
const PUBLIC_PREFIXES = ["NEXT_PUBLIC_", "VITE_", "REACT_APP_", "PUBLIC_", "EXO_PUBLIC_"];

for (const [name, value] of Object.entries(process.env)) {
  if (!value) continue;
  if (PUBLIC_PREFIXES.some(p => name.startsWith(p)) && FORBIDDEN.test(value)) {
    throw new Error(`FATAL: ${name} is client-exposed and holds a server secret.`);
  }
}
```

Wire this into the prebuild step. It converts a silent catastrophic misconfiguration into a failed build.

7.5 Rotation

Rotation is not only an incident response action. It is routine hygiene that limits the value of any credential an attacker may already hold undetected.

EVENT	ACTION	TIMING
Routine	Roll keys on a schedule	Quarterly is a reasonable default for live keys, or annually with strong compensating controls.
Someone with access leaves	Roll every key they could have read	Same day. Include keys in shared password managers and CI configurations.
Key appears in a commit	Roll immediately, then clean history	Within minutes. Assume it is already harvested; public repositories are scraped continuously.
Vendor or contractor engagement ends	Revoke the restricted key issued for it	At contract end. This is why third parties get their own restricted key and never yours.
Suspected compromise	Roll, then run the full runbook in 7.6	Immediately.

Table 7.4 Rotation triggers. Stripe's dashboard supports rolling a key with a short expiry window on the old value, which allows a deploy to complete before the previous key stops working.

PRACTICE

Rehearse rotation in test mode before you need it in production. The first time a team rolls a live key under incident pressure, they discover the key is hard-coded in a service nobody remembers, or cached in a long-lived process that will not pick up the new value. Find that out on a Tuesday afternoon, not during a breach.

7.6 The leaked key runbook

Write this down before you need it. During an incident, judgement degrades and a checklist outperforms improvisation.

STEP	ACTION	DETAIL
1	Roll the key	In the Stripe dashboard, immediately. Use a short expiry on the old key only if a deploy is genuinely required to swap it, and keep that window in minutes.
2	Deploy the new value	Update the secret manager, restart or redeploy everything that holds it. Confirm no service is still using the old key before it expires.
3	Audit the webhook endpoint list	An attacker with a secret key can register their own endpoint. This survives key rotation. Check it explicitly; almost nobody does.
4	Review API request logs	Stripe records every API call with source IP and key used. Look for requests from unexpected addresses, unusual object types, or bulk list operations.
5	Review money movement	Refunds, payouts, transfers, changes to bank account details, new coupons, modified prices. Compare against your own records rather than the dashboard alone.
6	Assess the data exposure	If bulk list calls occurred, treat it as a personal data breach and follow your notification obligations. This step is frequently skipped and is often the largest liability.
7	Purge the leak source	Rewrite git history or delete the repository, revoke the log, remove the artefact. Note that history rewriting does not retract anything already cloned or cached.
8	Contact Stripe support	Especially if fraudulent activity occurred. They can assist and they need to know.
9	Fix the cause	Add the detection that would have prevented it: secret scanning in CI, pre-commit hooks, the build guard from 7.4.

Table 7.5 Leaked key runbook. Steps 3 and 6 are the two most commonly omitted and the two with the longest tail.

7.7 Detection

Assume a secret will eventually be committed by someone tired at eleven at night. Build the net.

- **Pre-commit hooks.** `gitleaks` or `trufflehog` run locally, blocking the commit before it exists. Cheapest possible intervention point.
- **CI scanning.** The same tools in the pipeline, scanning full history on pull requests, because local hooks can be bypassed.
- **Provider push protection.** GitHub and equivalent platforms detect Stripe key patterns on push and can block it outright. Enable it.
- **Stripe's own detection.** Stripe scans public repositories for its key patterns and will notify you, and may revoke automatically. Do not treat this as your primary control; it is a last line, not a first.
- **Runtime egress awareness.** Alert on API requests to Stripe from source addresses outside your known infrastructure ranges.

7.8 The adjacent secret: your database service role

In modern stacks the Stripe key is rarely the only bearer credential in the system, and it is frequently not the most dangerous one. Backend-as-a-service platforms typically issue two keys: a public one intended for browsers and constrained by row level security, and a service role key that bypasses every policy.

CREDENTIAL	EXPOSURE	AUTHORITY
Anonymous or publishable key	Public	Constrained entirely by row level security policies. Safe in a browser only if RLS is enabled on every table and the policies are correct.
Service role key	Server only	Bypasses all row level security. Full read and write on every table. Equivalent to a database superuser reachable over the internet.

Table 7.6 Backend-as-a-service credentials. The service role key deserves the same handling as `sk_live_`, and it leaks more often because it is needed in webhook handlers, which are sometimes written quickly.

RULE

Row level security is enabled on every table that holds customer, subscription or entitlement data, and the policies are tested with the anonymous key, not merely written. A table without RLS in a project whose public key is in the browser is world-readable. Verify by querying it with the public key and confirming you get nothing back.

TRAP

Subscription and entitlement tables are the classic omission. The developer writes them from the webhook handler using the service role key, so RLS is never exercised during development and its absence is never noticed. Meanwhile the browser holds a key that can read the table directly, exposing every customer's plan, status and Stripe identifiers, and in careless schemas allowing a user to write their own tier.

CHAPTER EIGHT

The Money Path: Server-Authoritative Payments

One rule produces most of the correctness in a payment integration: the server decides what things cost, who is paying and what they are entitled to. The browser only expresses a wish.

This chapter is the centre of the book. Everything before it establishes context and everything after it handles the consequences, but the actual protection of your revenue happens here, in the few dozen lines that turn a request into a payment.

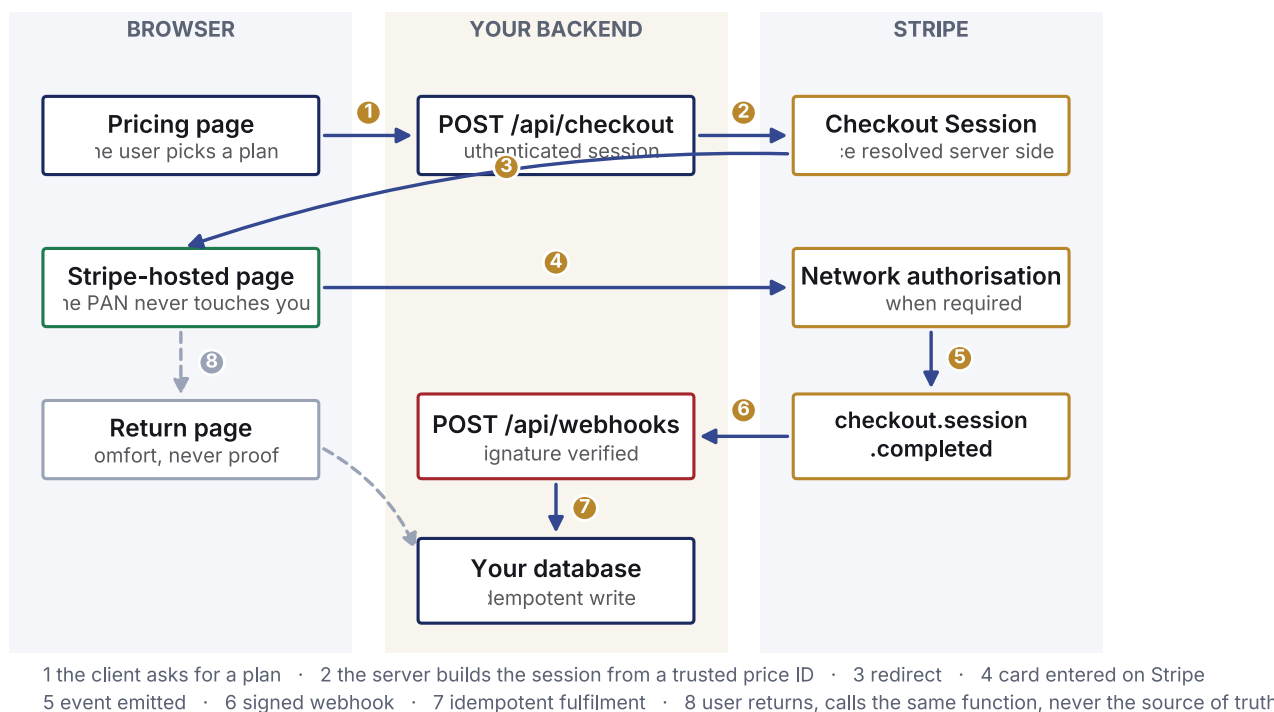


Figure 8.1 The reference flow. Note that the browser appears at three points and is authoritative at none of them. The dashed path is the user's return journey, which is a convenience and never a source of truth.

8.1 The price manipulation defect

Here is the shape of the bug, written the way it actually appears in codebases. It is readable, idiomatic, well typed, and it lets anyone buy anything for any price.

WRONG

```
// POST /api/checkout
export async function POST(req: Request) {
  const { amount, currency, productName, userId } = await req.json();

  const session = await stripe.checkout.sessions.create({
    mode: "payment",
    line_items: [{
      price_data: {
        currency,
        unit_amount: amount,           // from the browser
        product_data: { name: productName }, // from the browser
      },
      quantity: 1,
    }],
    metadata: { userId },              // from the browser
    success_url: `${BASE}/success`,
    cancel_url: `${BASE}/cancel`,
  });

  return Response.json({ url: session.url });
}
```

Three separate failures in one handler: the price, the product identity and the buyer identity all originate from an untrusted party.

The attack is not sophisticated. Open the network tab, find the request, change `amount` from `4900` to `1`, replay it. Stripe will happily create a session for one cent, because from Stripe's perspective the merchant asked for one cent. The webhook will fire with a genuine `checkout.session.completed`. The signature will verify. The fulfilment logic will grant the product. Every downstream control in this book will function correctly and the outcome will still be wrong, because the error was introduced before any of them ran.

The `userId` in metadata is the second, quieter failure. It lets an attacker pay one cent and credit the entitlement to any account they choose, including someone else's, which turns a pricing bug into an account takeover primitive.

RULE

Never construct `price_data` from request input. Reference a `price_id` that exists in Stripe and is selected from a table your server owns. Never accept an identity claim in a request body; derive the user from the authenticated session.

8.2 The corrected handler

RIGHT

```
// A server-owned catalogue. The client may name a key. It may not name a price.
const CATALOGUE = {
  starter_monthly: { priceId: "price_1QxStarterMonthly", mode: "subscription" },
  pro_monthly:     { priceId: "price_1QxProMonthly",     mode: "subscription" },
  pro_annual:      { priceId: "price_1QxProAnnual",      mode: "subscription" },
  lifetime:        { priceId: "price_1QxLifetime",       mode: "payment" },
} as const;

type PlanKey = keyof typeof CATALOGUE;

export async function POST(req: Request) {
  // 1. Identity comes from the session, never from the body.
  const user = await getAuthenticatedUser(req);
  if (!user) return new Response("Unauthorised", { status: 401 });

  // 2. The client names a plan. Validate it against the catalogue.
  const { plan } = await req.json();
  if (typeof plan !== "string" || !(plan in CATALOGUE)) {
    return new Response("Unknown plan", { status: 400 });
  }
  const entry = CATALOGUE[plan as PlanKey];

  // 3. Business rules the client cannot see, let alone influence.
  if (await hasActiveSubscription(user.id) && entry.mode === "subscription") {
    return new Response("Already subscribed", { status: 409 });
  }

  // 4. One Stripe customer per user, resolved server side and persisted.
  const customerId = await getOrCreateStripeCustomer(user);

  // 5. Create the session. Nothing here originated in the browser
  //     except the validated plan key.
  const session = await stripe.checkout.sessions.create({
    mode: entry.mode,
    customer: customerId,
    line_items: [{ price: entry.priceId, quantity: 1 }],
    client_reference_id: user.id,
    metadata: { userId: user.id, plan },
    success_url: `${BASE}/billing/return?session_id={CHECKOUT_SESSION_ID}`,
    cancel_url: `${BASE}/pricing`,
    automatic_tax: { enabled: true },
  }, {
    idempotencyKey: `checkout:${user.id}:${plan}:${dayBucket()}`,
  });

  return Response.json({ url: session.url });
}
```

The corrected handler. The only value crossing the trust boundary is a string checked against a fixed set of keys.

Five things changed, and each one closes a distinct hole:

	CHANGE	WHAT IT PREVENTS
1	Identity from the authenticated session	Paying for someone else's account, or crediting a purchase to an arbitrary user.
2	Plan key validated against a fixed catalogue	Arbitrary prices, arbitrary product names, currency substitution, injection of a price from another merchant.

	CHANGE	WHAT IT PREVENTS
3	Business rules applied server side	Double subscriptions, purchasing a plan the user is not eligible for, bypassing upgrade paths.
4	Customer resolved and persisted server side	Duplicate customer records, orphaned subscriptions, entitlements that cannot be reconciled later.
5	Idempotency key on the create call	Duplicate sessions from a double-clicked button or a retried request.

Table 8.1 The five corrections and the specific defect each one closes.

8.3 Quantity, and the negative number problem

Where quantity is genuinely user-selected, it is an untrusted integer and must be treated as one. Stripe rejects negative quantities, but your own logic often runs first, and a negative quantity that reaches a credit calculation before it reaches Stripe can grant balance rather than consume it.

```
function validateQuantity(raw: unknown, max = 20): number {
  if (typeof raw !== "number" || !Number.isInteger(raw)) {
    throw new BadRequest("Quantity must be an integer");
  }
  if (raw < 1 || raw > max) {
    throw new BadRequest(`Quantity must be between 1 and ${max}`);
  }
  return raw;
}
```

Reject rather than clamp. Silently coercing `-5` to `1` hides an attack you would rather see in your logs.

TRAP

Floating point amounts. Stripe works in the smallest currency unit, so EUR 49.00 is `4900`. Computing that as `Math.round(price * 100)` from a float introduces representation errors that appear as one-cent discrepancies in reconciliation and are extremely tedious to trace. Store prices as integers in minor units from the start, or use a decimal type. Also note the zero-decimal currencies: JPY and KRW among others take the amount directly, so `1000` means one thousand yen, not ten.

8.4 Discounts and coupons

Discount logic is where server authority is most often quietly abandoned, because promotion codes feel like user input. They are input, but the *discount* is not: the code is a lookup key and the reduction it produces is a server-side fact.

WRONG

```
const { discountPercent } = await req.json();
const finalAmount = basePrice * (1 - discountPercent / 100);
```

RIGHT

```
// Let Stripe own promotion codes entirely.
const session = await stripe.checkout.sessions.create({
  mode: "subscription",
  customer: customerId,
  line_items: [{ price: entry.priceId, quantity: 1 }],
  allow_promotion_codes: true, // Stripe validates, applies and enforces limits
  success_url: successUrl,
  cancel_url: cancelUrl,
});

// Or, when you must control eligibility yourself:
const code = await resolvePromotionCode(rawCode); // your lookup
if (!code || !(await isEligible(user, code))) {
  throw new BadRequest("Invalid or ineligible promotion code");
}
// pass discounts: [{ promotion_code: code.stripeId }]
```

Either Stripe owns the discount or your server does. The browser owns it in neither case.

8.5 Idempotency on outbound calls

Networks fail in the least convenient way: the request arrives, the operation completes, and the response is lost. Your client retries. Without an idempotency key, you have now created two customers, two subscriptions or two charges.

Stripe supports an idempotency key on POST requests. Send the same key with the same operation and you receive the original result rather than a second execution. Keys are retained for a bounded window, on the order of twenty-four hours, after which the same key is treated as new.

OPERATION	KEY CONSTRUCTION
Create Checkout Session	<code>checkout:{userId}:{plan}:{timeBucket}</code> so a double click reuses one session but a genuine second purchase later does not.
Create Customer	<code>customer:{userId}</code> . Stable forever, because there should only ever be one.
Refund	<code>refund:{chargeId}:{refundRequestId}</code> . Never a timestamp, or a retry becomes a second refund.
Subscription renewal charge	<code>renewal:{subscriptionId}:{periodStart}</code> . Naturally unique per billing period.
Transfer to a connected account	<code>payout:{accountId}:{payoutRunId}</code> . Derived from your own run identifier, not from wall-clock time.

Table 8.2 Idempotency key construction. The rule is that the key must be derived from the intent of the operation, so that a retry of the same intent produces the same key and a genuinely new operation produces a different one.

TRAP

Using a random UUID generated per attempt. This is the most common implementation and it provides no protection whatsoever, because the retry generates a new key and executes the operation again. If the key changes on retry, it is not an idempotency key, it is a request ID.

8.6 Metadata as your correlation layer

Stripe objects and your database rows need to be joinable in both directions, at three in the morning, from a support ticket that contains only an email address. Metadata is how you make that possible.

```
metadata: {  
  userId: user.id,           // your primary key, not an email  
  orgId: user.orgId ?? "",  
  plan: plan,  
  appVersion: RELEASE_SHA,   // which build created this  
  source: "web_pricing_page",  
}
```

- Always include your own primary key. Emails change; identifiers should not.
- Never put anything sensitive in metadata. It is visible to everyone with dashboard access and it appears in exports.
- Metadata is a correlation aid, never an authorisation input. Read Chapter 9 before you trust a metadata value that arrived on a webhook.
- Use `client_reference_id` on Checkout Sessions as well; it is indexed and searchable in the dashboard.

8.7 Client secrets

When you drive confirmation yourself with the Payment Element, the browser receives a client secret. It is scoped to a single intent, but it does confer the ability to confirm and read that intent, so it needs ordinary care.

- Return it only over an authenticated request, from a server that has verified the intent belongs to the requesting user.
- Never place it in a URL, where it lands in browser history, referrer headers and server access logs.
- Never log it, and never store it beyond the life of the page.
- Do not treat possession of a client secret as evidence of payment. It is evidence of an attempt.

Every question of the form "can the client be trusted with this" has the same answer. The interesting question is only ever how to arrange the code so that it never needs to be.

CHAPTER NINE

Webhooks: The Load-Bearing Wall

A webhook endpoint is a public HTTP handler that anyone can call, and it hands out what people paid for. Everything in this chapter follows from that.

Webhooks are how Stripe tells you what happened. A payment succeeded, a subscription renewed, a dispute opened, a payout failed. Because payments are asynchronous, and because a customer's browser can close at any moment, the webhook is the only delivery mechanism that reliably reaches you. It is therefore the component on which the correctness of your entire billing state depends.

It is also the component most frequently implemented wrong, and Figure 1.3 quantified how frequently: around a quarter of publicly reachable Stripe webhook endpoints in one large scan performed no signature verification at all.

9.1 What an unverified endpoint gives away

Suppose your handler looks like this. It is short, it works, and it is a free subscription dispenser.

WRONG

```
export async function POST(req: Request) {
  const event = await req.json();           // no verification at all

  if (event.type === "checkout.session.completed") {
    const s = event.data.object;
    await grantAccess(s.metadata.userId, s.metadata.plan);
  }
  return new Response("ok");
}
```

The attack requires no tooling beyond curl. The endpoint URL is typically `/api/webhooks/stripe` or `/api/stripe`, and if it is not guessable it is usually in a public repository or discoverable through a JavaScript source map.

```
curl -X POST https://target.example.com/api/webhooks/stripe \
-H 'Content-Type: application/json' \
-d '{"type":"checkout.session.completed",
  "data":{"object":{"metadata":{"userId":"attacker-id","plan":"lifetime"}}}}'
```

A complete exploit. Free lifetime access, repeatable, with no payment and no account.

RULE

Verify the signature before parsing the body, before logging, before touching a database, before anything. Verification is the first statement in the handler and there is no code path that reaches business logic without passing it.

9.2 How the signature works

Stripe sends a `Stripe-Signature` header containing a timestamp and one or more signatures. The signature is an HMAC using SHA-256, computed over the timestamp concatenated with the raw request body, keyed with the endpoint's signing secret.

```
Stripe-Signature: t=1758000000,
                  v1=5257a869e7ecebeda32affa62cdca3fa51cad7e77a0e56ff536d0ce8e108d8bd,
                  v1=a0b1c2... // a second signature appears during secret rotation
```

Verification, conceptually, is four steps:

1. Split the header and extract `t` and every `v1` value.
2. Build the signed payload as `t`, a full stop, then the **raw request body bytes**.
3. Compute `HMAC-SHA256(signedPayload, endpointSecret)` and compare, in constant time, against each supplied `v1`.
4. Reject if the timestamp is outside your tolerance window, which defaults to five minutes in Stripe's libraries.

You should not implement this yourself. The official libraries do it correctly, including the constant-time comparison and the multi-signature case that arises during secret rotation.

```
import Stripe from "stripe";
const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!);
const SECRET = process.env.STRIPE_WEBHOOK_SECRET!;

export async function POST(req: Request) {
  const signature = req.headers.get("stripe-signature");
  if (!signature) return new Response("Missing signature", { status: 400 });

  const raw = await req.text(); // RAW body. See 9.3.

  let event: Stripe.Event;
  try {
    event = stripe.webhooks.constructEvent(raw, signature, SECRET);
  } catch (err) {
    // Do not log the body. Do not leak the reason to the caller.
    logger.warn("webhook signature verification failed");
    return new Response("Invalid signature", { status: 400 });
  }

  // Only now does the event exist as far as this system is concerned.
  await enqueue(event);
  return new Response(null, { status: 200 });
}
```

The canonical handler. Note that it returns 200 after enqueueing, not after processing. Section 9.6 explains why.

9.3 The raw body problem

This is the defect that turns a correctly written verification into a broken one, and it is subtle enough that it survives code review.

The signature is computed over the exact bytes Stripe sent. If any middleware parses the JSON and your handler then re-serialises it, the bytes change. Key order, whitespace, unicode escaping, numeric formatting: any difference produces a different HMAC and verification fails. The usual outcome is a developer who cannot get verification to pass, and who resolves it by removing verification.

FRAMEWORK	HOW TO OBTAIN THE RAW BODY
Express	<code>express.raw({ type: "application/json" })</code> mounted on the webhook route before any global <code>express.json()</code> .
Next.js App Router	<code>await req.text()</code> . Works by default; do not call <code>req.json()</code> first.
Next.js Pages Router	Export <code>export const config = { api: { bodyParser: false } }</code> and read the stream manually.
Fastify	Register a content type parser that preserves the raw buffer for the webhook route.
Django	<code>request.body</code> , and exempt the view from CSRF.
Flask	<code>request.get_data()</code> , not <code>request.get_json()</code> .
Rails	<code>request.body.read</code> , with CSRF protection skipped for the action.
Serverless and edge	Check whether the platform base64-encodes the body. If it does, decode before verifying.

Table 9.1 Obtaining the unmodified request body. Getting this wrong is the most common reason teams disable verification.

TRAP

A body-size limit that truncates large events. Some events, particularly invoices with many line items, exceed default limits. The body is silently truncated, the signature fails, and the failure is intermittent and correlated with data size, which makes it look like a Stripe problem. Raise the limit for the webhook route specifically.

9.4 The full validation pipeline

Signature verification is necessary and not sufficient. A complete handler passes an event through seven gates before it is allowed to change anything.

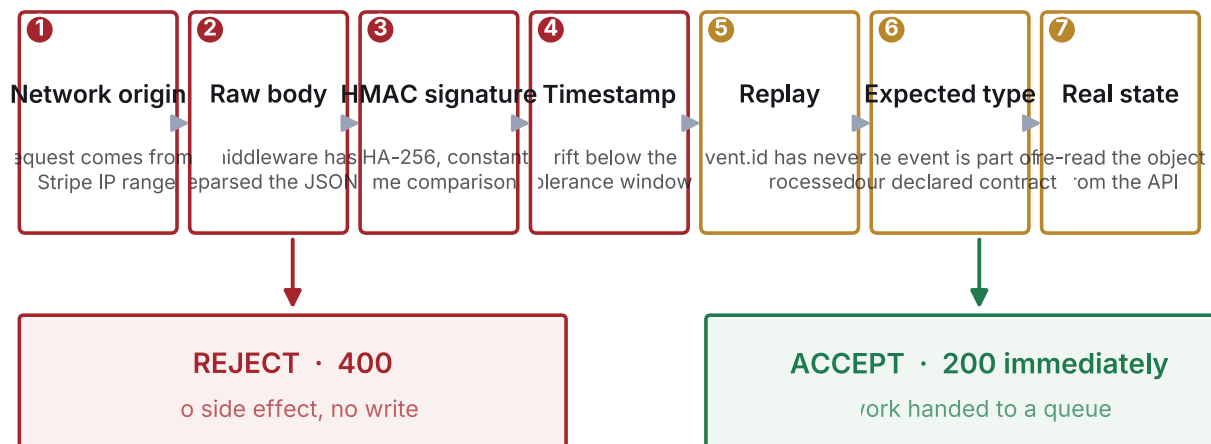


Figure 9.1 The seven gates. The first four are authenticity, the last three are correctness. An event that passes all seven may change state; anything else is rejected or acknowledged and discarded.

GATE	CHECK	WHY
1	Network origin	Optional defence in depth. Stripe publishes its webhook source ranges. Useful, but never a substitute for signatures, since IP ranges change and can be spoofed in some network positions.
2	Raw body preserved	Section 9.3. Without it, gate 3 cannot work.
3	HMAC signature valid	The actual authenticity control. Constant-time comparison against every supplied signature.
4	Timestamp within tolerance	Bounds the replay window for a captured event. Handled by the library's default tolerance; do not widen it casually.
5	Event ID not seen before	Stripe retries, and at-least-once delivery is the contract. Without this, retries duplicate side effects. Section 9.5.
6	Event type is one you handle	Ignore unknown types with a 200. Do not fall through to a generic handler, and do not error, or Stripe will retry an event you were never going to process.
7	Re-read the object from the API	The event payload is a snapshot from when the event fired. Events can arrive out of order, so the object may have changed. For anything consequential, fetch current state before acting.

Table 9.2 The seven gates in detail. Gates 5 and 7 are the ones that separate a working integration from a correct one.

CONTEXT

Gate 7 deserves emphasis because it is counter-intuitive. Stripe explicitly does not guarantee event ordering. A `customer.subscription.updated` event describing a downgrade can arrive after a later event describing a re-upgrade. If you apply payload snapshots blindly, you will apply them in the wrong order and end up with state that never existed. Re-reading the object, or comparing a monotonic field before applying, resolves this.

9.5 Idempotency on the inbound side

Stripe guarantees at-least-once delivery and retries on failure for up to several days with increasing intervals. It will also occasionally deliver the same event twice under normal operation. Your handler must therefore be safe to run repeatedly with the same input.

The mechanism is the event ID, which is stable across every delivery of the same event. Record it before doing work, in the same transaction as the work itself.

```
-- Deduplication table. The primary key does the work.
create table stripe_events (
  id          text primary key,      -- evt_...
  type        text      not null,
  received_at timestamptz not null default now(),
  processed_at timestamptz,
  attempts    int        not null default 0,
  last_error   text
);
```

```
async function processEvent(event: Stripe.Event) {
  // Claim the event. If another delivery already claimed it, stop.
  const claimed = await db.transaction(async (tx) => {
    const inserted = await tx.query(
      `insert into stripe_events (id, type) values ($1, $2)
       on conflict (id) do nothing
       returning id`,
      [event.id, event.type],
    );
    return inserted.rowCount === 1;
  });

  if (!claimed) {
    logger.info({ eventId: event.id }, "duplicate delivery ignored");
    return;
  }

  try {
    await handle(event);                // your business logic
    await db.query(
      `update stripe_events set processed_at = now() where id = $1`,
      [event.id],
    );
  } catch (err) {
    await db.query(
      `update stripe_events
       set attempts = attempts + 1, last_error = $2
       where id = $1`,
      [event.id, String(err)],
    );
    throw err;                          // let the queue retry
  }
}
```

Claim-then-process. The unique constraint is the entire deduplication mechanism, and it works correctly under concurrent delivery.

TRAP

Checking for existence and then inserting as two separate statements. Two concurrent deliveries both read "not present", both proceed, and both grant the entitlement. Use a single atomic insert with a conflict clause, or a unique constraint that raises. The check-then-act pattern is a race condition regardless of how fast the two statements are.

9.6 Respond fast, work later

Stripe expects a timely response, on the order of seconds, and treats a slow endpoint as a failed delivery. If your handler provisions infrastructure, sends email, calls three internal services and writes to a warehouse, it will time out, Stripe will retry, and you will process the event twice while the first attempt is still running.

RULE

The webhook handler verifies, deduplicates, persists and returns 200. All actual work happens in a queue consumer. The HTTP response acknowledges receipt of the event; it does not report the outcome of the work.

```
export async function POST(req: Request) {
  const sig = req.headers.get("stripe-signature");
  const raw = await req.text();

  let event: Stripe.Event;
  try {
    event = stripe.webhooks.constructEvent(raw, sig!, SECRET);
  } catch {
    return new Response("Invalid signature", { status: 400 });
  }

  if (!HANDLED_TYPES.has(event.type)) {
    return new Response(null, { status: 200 }); // acknowledged, ignored
  }

  try {
    await eventQueue.enqueue({ id: event.id, type: event.type });
  } catch {
    // We could not even record it. Ask Stripe to try again.
    return new Response("Queue unavailable", { status: 500 });
  }

  return new Response(null, { status: 200 });
}
```

What each status code means to Stripe

YOU RETURN	STRIPE DOES	USE IT WHEN
2xx	Marks delivered, stops retrying	You have durably recorded the event, whether or not you have processed it.

YOU RETURN	STRIPE DOES	USE IT WHEN
400	Marks failed, retries per schedule	Signature verification failed. The retry will also fail, which is correct: the event was not from Stripe.
4xx other	Marks failed, retries	Rarely appropriate. A 404 usually means a misconfigured endpoint URL.
5xx	Marks failed, retries with backoff	Your infrastructure is temporarily unable to record the event and you want it redelivered.
Timeout	Marks failed, retries	Never intentionally. Fix the handler.

Table 9.3 Response semantics. The key asymmetry: 200 means "I have it", not "I have finished with it". Returning 500 because your email provider is down causes Stripe to redeliver a payment event you already recorded.

9.7 Endpoint hygiene

- **One signing secret per endpoint.** They are not interchangeable. A single handler serving both a platform endpoint and a Connect endpoint needs to select the right secret, usually by route.
- **Subscribe to specific event types.** Selecting all events means your endpoint receives categories you never handle, adds noise, and increases the chance of a handler falling through into logic meant for something else.
- **Separate endpoints for separate concerns.** Billing events and Connect account events have different handlers, different failure modes and different on-call owners.
- **Audit the endpoint list quarterly.** An attacker with a secret key can add an endpoint, and that access survives rotating the key they used. This is in the runbook in 7.6 for a reason.
- **Alert on delivery failure.** Stripe surfaces failures in the dashboard and can notify you. A webhook endpoint that has been failing quietly for a week is a week of billing state you do not have.
- **Keep the payload out of your logs.** Event bodies contain personal data. Log the event ID and type; fetch the rest when you need it.

9.8 Testing the endpoint properly

The Stripe CLI forwards real test-mode events to a local endpoint and can trigger specific event types on demand.

```
stripe login
stripe listen --forward-to localhost:3000/api/webhooks/stripe
# prints a whsec_ for this session; use it as your local secret

stripe trigger checkout.session.completed
stripe trigger invoice.payment_failed
stripe trigger customer.subscription.deleted
```

Beyond the happy path, your test suite should assert the security properties directly, because those are exactly the ones a manual click-through never exercises:

- ☐ A request with no `Stripe-Signature` header is rejected with 400.
- ☐ A request with a signature computed using the wrong secret is rejected.
- ☐ A request with a valid signature but a timestamp an hour old is rejected.
- ☐ The same valid event delivered twice produces exactly one entitlement change.
- ☐ Ten concurrent deliveries of the same event produce exactly one entitlement change.
- ☐ An unhandled event type returns 200 and changes nothing.
- ☐ A downstream failure during processing does not mark the event processed.

If your test suite contains only the first of those, you have tested that Stripe works. You have not tested your integration.

CHAPTER TEN

Fulfilment and Data Consistency

The payment succeeded. Now something has to happen in your product, exactly once, whether or not the customer stayed for the redirect.

Fulfilment is the act of granting what was paid for: activating a subscription, shipping an order, adding credits, unlocking a feature, provisioning a tenant. It is the last step of the money path and the first step where your product's own semantics take over.

Two failure modes bracket the problem. Fulfil too eagerly and you give away product to people who did not pay. Fulfil too conservatively and paying customers sit locked out while support tickets accumulate. The correct design avoids both, and it turns out to be simpler than most teams expect.

10.1 The return page is not evidence

The most common fulfilment bug is trusting the redirect. It looks reasonable: Stripe sends the customer to `success_url`, therefore the payment succeeded, therefore grant access.

WRONG

```
// GET /billing/success
export default async function SuccessPage({ searchParams }) {
  await grantAccess(currentUser.id, searchParams.plan); // never do this
  return <h1>Thank you</h1>;
}
```

Anyone can navigate to `/billing/success?plan=lifetime`. There is no payment involved, no session identifier, no verification. This is not an edge case; it is a URL.

Even the more careful version, where the page reads a `session_id` query parameter and looks it up, must not be your *only* fulfilment path, because the return journey is unreliable in ways that have nothing to do with attackers:

- The customer closes the tab immediately after paying.
- The redirect fails, or a network drops during it.
- The payment method settles asynchronously, so the customer returns before the money does. Bank debits, vouchers and several regional methods behave this way.
- The payment was made from a link opened on a different device.

RULE

The webhook is the authoritative fulfilment trigger. The return page may call the same fulfilment function to make the experience feel immediate, but it must never be the only thing that does, and it must verify the session server side before it does anything.

10.2 One idempotent function, two callers

The pattern that resolves all of this is a single fulfilment function, safe to run any number of times, invoked from both the webhook and the return page. Stripe's own documentation recommends exactly this structure, and it is worth adopting verbatim.

```
/**
 * Idempotent. Safe to call from the webhook, from the return page,
 * from a reconciliation job, and from a support tool, in any order,
 * any number of times.
 */
export async function fulfilCheckout(sessionId: string): Promise<void> {
  // 1. Read the real object from Stripe. Never trust a passed-in payload.
  const session = await stripe.checkout.sessions.retrieve(sessionId, {
    expand: ["line_items", "payment_intent"],
  });

  // 2. Confirm money actually moved. "complete" alone is not enough:
  //    a session can be complete with payment still pending.
  const paid = session.payment_status === "paid"
    || session.payment_status === "no_payment_required";
  if (session.status !== "complete" || !paid) {
    logger.info({ sessionId, status: session.status }, "not fulfilable yet");
    return;
  }

  // 3. Resolve the buyer from data we control, not from the browser.
  const userId = session.client_reference_id ?? session.metadata?.userId;
  if (!userId) {
    logger.error({ sessionId }, "session has no resolvable user");
    throw new Error("Unresolvable session");
  }

  // 4. Claim the fulfilment atomically. This is the idempotency barrier.
  const claimed = await db.query(
    `insert into fulfilments (checkout_session_id, user_id, status)
     values ($1, $2, 'in_progress')
     on conflict (checkout_session_id) do nothing
     returning id`,
    [sessionId, userId],
  );
  if (claimed.rowCount === 0) return; // already fulfilled or in flight

  // 5. Do the work.
  try {
    await grantEntitlements(userId, session);
    await db.query(
      `update fulfilments set status = 'done', completed_at = now()
       where checkout_session_id = $1`, [sessionId]);
    await sendReceipt(userId, session); // side effects that may fail safely
  } catch (err) {
    await db.query(
      `update fulfilments set status = 'failed', last_error = $2
       where checkout_session_id = $1`, [sessionId, String(err)];
    throw err;
  }
}
```

The reference fulfilment function. Steps 1, 2 and 4 are the security properties; steps 3 and 5 are the business logic.

Now both callers are trivial, and neither can be exploited:

```
// Caller A: the webhook. Authoritative.
case "checkout.session.completed":
case "checkout.session.async_payment_succeeded":
  await fulfilCheckout(event.data.object.id);
  break;

case "checkout.session.async_payment_failed":
  await recordFailedPayment(event.data.object.id);
  break;
```

```
// Caller B: the return page. Cosmetic, but still verified.
export default async function ReturnPage({ searchParams }) {
  const user = await requireAuthenticatedUser();
  const sessionId = searchParams.session_id;
  if (!sessionId) redirect("/pricing");

  const session = await stripe.checkout.sessions.retrieve(sessionId);

  // The session must belong to this user. Otherwise it is someone
  // else's session ID pasted into a URL.
  if (session.client_reference_id !== user.id) {
    redirect("/billing");
  }

  await fulfilCheckout(sessionId); // idempotent, so this is free
  return <ThankYou session={session} />;
}
```

The ownership check on the return page is not optional. Without it, a session ID from any customer grants that customer's entitlement to whoever pastes it.

10.3 Payment status is not one boolean

A Checkout Session carries both a `status` and a `payment_status`, and conflating them causes a specific and expensive class of bug with asynchronous payment methods.

FIELD	VALUES	MEANING
<code>status</code>	<code>open</code> , <code>complete</code> , <code>expired</code>	Whether the customer finished the checkout flow. Says nothing about money.
<code>payment_status</code>	<code>paid</code> , <code>unpaid</code> , <code>no_payment_required</code>	Whether funds are actually secured. <code>no_payment_required</code> covers zero-amount cases such as a fully discounted trial.

Table 10.1 A session can be `complete` and `unpaid` simultaneously. That combination is normal for delayed notification payment methods, and fulfilling on it means shipping goods against money that has not arrived and may never arrive.

TRAP

Enabling additional payment methods later, after the fulfilment logic was written and tested against cards only. Cards settle synchronously, so `complete` and `paid` always coincide during development. Add a bank debit or a voucher-based method and suddenly sessions complete with payment pending. The code that worked for two years starts giving product away. This is why step 2 in `fulfilCheckout` checks both fields explicitly rather than assuming.

10.4 Modelling subscription state

For recurring billing, the question is where entitlement lives. Three approaches, and only one of them is workable at scale.

APPROACH	HOW IT WORKS	ASSESSMENT
Query Stripe on every request	Ask the API whether the user has an active subscription each time they load a page.	Correct but unusable. Adds latency to every request, consumes rate limit, and fails closed when Stripe has an incident, locking out every paying customer.
Mirror in your database, updated by webhooks	Maintain a local table of subscription state, updated whenever a relevant event arrives.	The right answer. Fast, resilient, queryable, and joinable with your own data. Requires disciplined event handling and a reconciliation job.
Set a flag at purchase and never revisit	Grant access on <code>checkout.session.completed</code> and forget.	Wrong within one billing cycle. Cancellations, failed renewals, downgrades and disputes never propagate, so cancelled customers keep access indefinitely.

Table 10.2 Entitlement storage strategies. The mirror is the only one that is both fast and eventually correct.

```
create table subscriptions (
  id                text primary key,          -- sub_...
  user_id           uuid not null references users(id),
  stripe_customer_id text not null,
  status            text not null,             -- Stripe's status verbatim
  price_id          text not null,
  current_period_end timestampz not null,
  cancel_at_period_end boolean not null default false,
  updated_at        timestampz not null default now(),
  -- Guards against applying a stale, out-of-order event:
  last_event_created_at timestampz not null
);

create unique index on subscriptions (user_id) where status in ('active','trialing');
create index on subscriptions (stripe_customer_id);
```

Store Stripe's status verbatim rather than a derived boolean. Deriving entitlement from status at read time keeps one source of truth; storing a boolean creates two that will disagree.

```
// Apply only if this event is newer than what we already stored.
await db.query(
  `insert into subscriptions
    (id, user_id, stripe_customer_id, status, price_id,
     current_period_end, cancel_at_period_end, last_event_created_at)
  values ($1,$2,$3,$4,$5,$6,$7,$8)
  on conflict (id) do update set
    status          = excluded.status,
    price_id        = excluded.price_id,
    current_period_end = excluded.current_period_end,
    cancel_at_period_end = excluded.cancel_at_period_end,
    updated_at      = now(),
    last_event_created_at = excluded.last_event_created_at
  where subscriptions.last_event_created_at < excluded.last_event_created_at`,
  [...])
);
```

The `where` clause on the update is the out-of-order guard from gate 7 in Chapter 9, expressed in SQL. An older event arriving late is silently discarded.

Which status values grant access

STATUS	ACCESS	MEANING AND HANDLING
<code>trialing</code>	Yes	In a trial period. Full access.
<code>active</code>	Yes	Paid and current.
<code>past_due</code>	Your choice	A renewal failed and Stripe is retrying. Most businesses keep access during the retry window and communicate. Cutting off immediately converts a recoverable card expiry into a churned customer.
<code>unpaid</code>	No	Retries exhausted. Revoke, but keep the data.
<code>canceled</code>	No	Ended. Note that <code>cancel_at_period_end</code> means access continues until <code>current_period_end</code> while status is still <code>active</code> .
<code>incomplete</code>	No	The first payment never completed, often awaiting authentication.
<code>incomplete_expired</code>	No	The first payment was never completed in time. Terminal.
<code>paused</code>	No	Collection paused, typically after a trial ended without a payment method.

Table 10.3 Subscription statuses and entitlement. Encode this table in one function used everywhere, rather than scattering status comparisons through the codebase.

10.5 Reconciliation

Webhooks are reliable but not infallible. Endpoints go down, deploys drop events, bugs cause handlers to fail silently, and Stripe eventually stops retrying. Over a long enough period your mirror *will* drift from reality.

PRACTICE

Run a scheduled reconciliation job. Nightly is ample. List subscriptions modified since the last run, compare each against your local row, correct differences, and alert on any correction, because a correction means a webhook was missed and you want to know why. This job is perhaps sixty lines and it converts an entire class of silent billing incident into a monitored one.

```
export async function reconcileSubscriptions(since: Date) {
  let drift = 0;
  for await (const sub of stripe.subscriptions.list({
    created: { gte: Math.floor(since.getTime() / 1000) },
    limit: 100,
    expand: ["data.customer"],
  })) {
    const local = await getLocalSubscription(sub.id);
    if (!local || local.status !== sub.status ||
        local.price_id !== sub.items.data[0]?.price.id) {
      await upsertSubscription(sub);
      drift++;
      logger.warn({ subscriptionId: sub.id }, "reconciliation corrected drift");
    }
  }
  metrics.gauge("billing.reconciliation.drift", drift);
}
```

10.6 Refunds, disputes and revocation

Granting access is only half of fulfilment. Revocation is the half that gets forgotten, and the omission is expensive because it is invisible: nothing breaks, revenue simply leaks.

EVENT	WHAT MUST HAPPEN
<code>charge.refunded</code>	Revoke or partially revoke the entitlement bought by that charge. Decide explicitly whether partial refunds partially revoke.
<code>charge.dispute.created</code>	Revoke immediately in most models, since funds are withdrawn pending resolution. Flag the account for review.
<code>customer.subscription.deleted</code>	Revoke at the appropriate boundary. Respect <code>cancel_at_period_end</code> semantics.
<code>invoice.payment_failed</code>	Begin dunning. Do not revoke on the first failure unless your model requires it.
<code>customer.subscription.updated</code>	Apply plan changes, including downgrades that reduce limits. Downgrades are where entitlement bugs hide.

Table 10.4 Revocation triggers. Every grant path needs a matching revoke path, and the revoke path needs a test.

Write the revocation test first. It is the one nobody writes and the one that catches the bug that costs money quietly for a year.

CHAPTER ELEVEN

Rate Limits, Errors and Resilience

Payment infrastructure fails the way every distributed system fails. The difference is that the failures are counted in money, and a careless retry can charge someone twice.

11.1 The limits

Stripe applies rate limits per account, and separately per endpoint. They exist to protect the platform and, indirectly, to protect you from a runaway loop billing your customers.

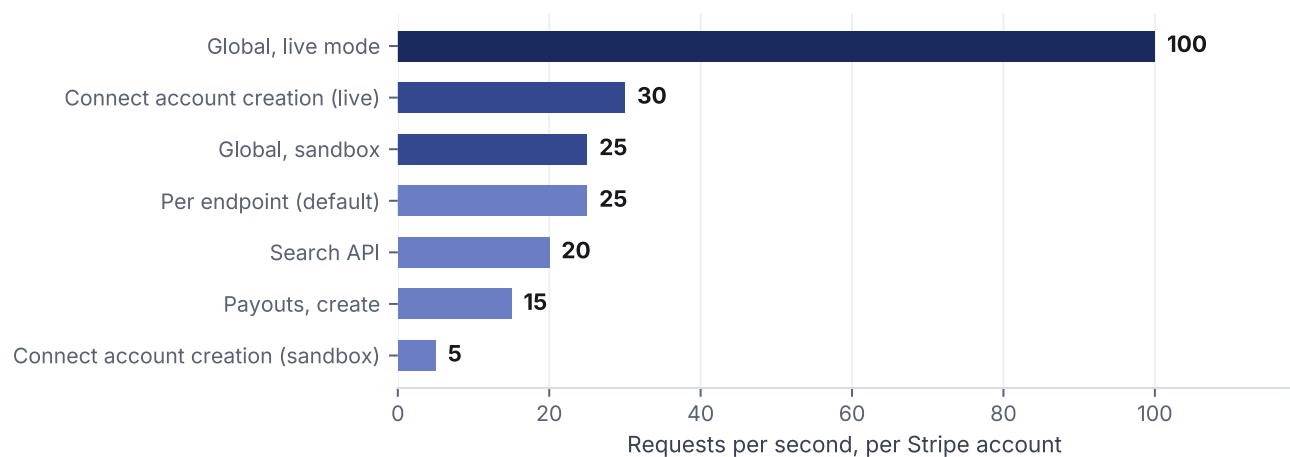


Figure 11.1 Documented rate limits by category. Sandbox limits are deliberately lower than live limits, which means a load test in sandbox will hit ceilings that production would not. Verify current figures in Stripe's documentation before designing around them.

Two consequences worth internalising. First, limits are per account, not per key, so a reporting job that lists every charge can starve your checkout endpoint. Second, exceeding a limit returns HTTP 429 with a `rate_limit` error, which is retryable, but only with backoff.

PRACTICE

Isolate bulk work from interactive work. Batch jobs, exports and reconciliation should run on a schedule that avoids peak checkout traffic, use their own restricted key so their consumption is attributable, and back off aggressively. Where Stripe offers a bulk mechanism, such as Sigma for analytical queries or the reporting API, use it instead of paginating the live API.

11.2 The error taxonomy

Stripe's errors carry a type, and the type tells you what to do. Treating all errors identically is the root of most bad retry behaviour.

TYPE	RETRY	MEANING AND CORRECT HANDLING
<code>card_error</code>	No	The card was declined. Surface the decline message to the customer and let them try another method. Retrying identically will decline identically.
<code>invalid_request_error</code>	No	Your request was malformed or referenced something that does not exist. A bug in your code. Alert, do not retry.
<code>authentication_error</code>	No	The API key is wrong, revoked or expired. Page someone. This is a production outage.
<code>permission_error</code>	No	A restricted key lacks the scope for this call. Configuration problem, not a transient one.
<code>rate_limit_error</code>	Yes	Back off exponentially with jitter. Consider whether the calling pattern is the real problem.
<code>api_error</code>	Yes	Something went wrong on Stripe's side. Rare. Retry with backoff and an idempotency key.
<code>api_connection_error</code>	Yes, carefully	The request may or may not have been received. Always retry with the same idempotency key, or you risk a duplicate charge.
<code>idempotency_error</code>	No	The same idempotency key was reused with different parameters. A bug in key construction. See Table 8.2.

Table 11.1 Stripe error types. The distinction between retryable and not is the important column, and `api_connection_error` is the dangerous one.

RULE

Never retry a write operation without an idempotency key. A connection error means you do not know whether the charge happened. Retrying without a key resolves that uncertainty by creating a second charge.

11.3 Backoff that behaves

Naive retries make outages worse. When a dependency recovers, every client that has been retrying on a fixed interval hits it simultaneously and knocks it over again. Exponential backoff with jitter spreads the load.

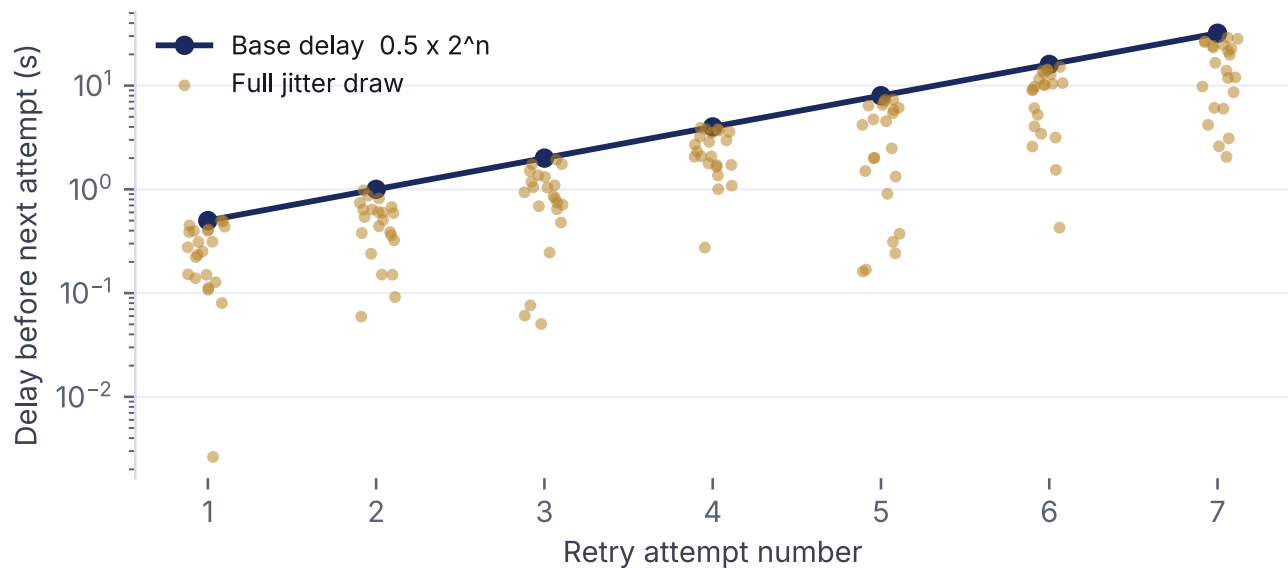


Figure 11.2 Exponential backoff with full jitter. The solid line is the deterministic delay; the scatter shows the randomised draws. Randomisation is what prevents synchronised retry storms.

```

async function withRetry<T>({
  op: (attempt: number) => Promise<T>,
  { maxAttempts = 5, baseMs = 500, capMs = 20_000 } = {},
}): Promise<T> {
  let lastError: unknown;

  for (let attempt = 1; attempt <= maxAttempts; attempt++) {
    try {
      return await op(attempt);
    } catch (err: any) {
      lastError = err;

      const retryable = ["rate_limit_error", "api_error", "api_connection_error"]
        .includes(err?.type);
      if (!retryable || attempt === maxAttempts) throw err;

      // Honour an explicit Retry-After if Stripe sent one.
      const retryAfter = Number(err?.headers?.["retry-after"]);
      const exponential = Math.min(capMs, baseMs * 2 ** (attempt - 1));
      const delay = Number.isFinite(retryAfter) && retryAfter > 0
        ? retryAfter * 1000
        : Math.random() * exponential; // full jitter

      await sleep(delay);
    }
  }
  throw lastError;
}

// Usage: the idempotency key is fixed across attempts, deliberately.
const key = `refund:${chargeId}:${refundRequestId}`;
await withRetry(() =>
  stripe.refunds.create({ charge: chargeId }, { idempotencyKey: key }));

```

Note that the key is computed once, outside the retry loop. Computing it inside would defeat the entire mechanism.

CONTEXT

The official Stripe SDKs include built-in retry support with idempotency handling, configured through `maxNetworkRetries`. For most applications, enabling it is preferable to writing your own loop. Write your own only when you need behaviour the SDK does not offer, such as coordinating with a circuit breaker or a shared token bucket across processes.

11.4 Timeouts, circuit breakers and degradation

Set explicit timeouts. The default in many HTTP clients is effectively unbounded, which means a slow dependency converts into exhausted connection pools and a full outage.

```
const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  timeout: 20_000,           // ms; fail rather than hang
  maxNetworkRetries: 2,      // SDK-managed, idempotency-aware
  telemetry: false,
});
```

When Stripe is genuinely unavailable, decide in advance how your product behaves. The important insight is that different operations deserve different answers.

OPERATION	ON FAILURE	REASONING
Creating a new payment	Fail visibly	Tell the customer clearly and invite them to retry. Never queue a payment silently to be charged later; that produces charges customers do not expect.
Checking entitlement	Serve from the local mirror	This is the entire justification for Chapter 10's mirror. Paying customers keep working through a Stripe incident.
Processing a webhook	Return 500 and let Stripe retry	Redelivery is a designed-in feature. Use it.
Reporting and analytics	Defer	Nobody is harmed by a delayed report. Back off hard and free the capacity.
Cancelling a subscription	Queue and confirm	Record the customer's intent locally, apply it when the API returns, and honour it in your own entitlement logic immediately. Never make a customer fight to cancel.

Table 11.2 Degradation strategy by operation. "Fail the whole application" is almost never the right answer, and neither is "pretend it worked".

11.5 What to monitor

Payment systems need monitoring that is different in kind from ordinary service monitoring, because the dangerous failures do not raise errors.

SIGNAL	ALERT WHEN	CATCHES
Webhook delivery failure rate	Any sustained failure	A broken endpoint, a bad deploy, an expired secret.

SIGNAL	ALERT WHEN	CATCHES
Events received but never processed	Age exceeds a threshold	A stuck queue, a crashing consumer, a handler exception swallowed silently.
Payments succeeded without matching fulfilment	Any occurrence	The most valuable alert in the entire system. Customers who paid and got nothing.
Fulfilments without a matching payment	Any occurrence	The inverse: product given away. Often the first sign of a forged or mishandled event.
Reconciliation drift count	Above zero	Missed webhooks, out-of-order application, handler bugs.
Authorisation rate	Drops by a few points	Card testing, a misconfigured Radar rule, a payment method problem, an issuer change.
Dispute rate	Approaching network thresholds	Fraud, a confusing billing descriptor, a product problem. Chapter 13.
4xx and 5xx from the Stripe API by type	Any change in shape	Deploys that break request construction, keys losing scope, version drift.
Rate limit responses	Any sustained occurrence	A job with a runaway loop, or growth that has outpaced your calling pattern.

Table 11.3 The monitoring set. The two reconciliation alerts, rows three and four, are the ones that would have caught nearly every incident I have been asked to investigate after the fact.

PRACTICE

Build a daily reconciliation report that answers one question: does the set of successful payments in Stripe exactly match the set of fulfilments in your database for the same period? Report both directions of the difference. If the answer is ever anything other than "yes, zero discrepancies", you want to find out on the day it happens rather than during an audit.

PART FOUR

Fraud, Risk and Disputes

Some payments should not be accepted, and some accepted payments will be taken back. Both are managed rather than eliminated, and both are as much a product problem as a security one.

12 Radar and Transaction Risk

13 Disputes, Chargebacks and Evidence

CHAPTER TWELVE

Radar and Transaction Risk

Fraud prevention is a tuning problem with money on both sides: the fraud you let through, and the good customers you turn away, which is usually the bigger number. This chapter is about where to set the dials.

Stripe Radar scores every payment using a model trained across the platform's aggregate traffic, which gives it visibility no single merchant could reproduce: a card that has just been tested against dozens of unrelated businesses looks different to Radar than it does to you.

12.1 Scores and levels

Each payment receives a risk score from 0 to 99, mapped onto a risk level.

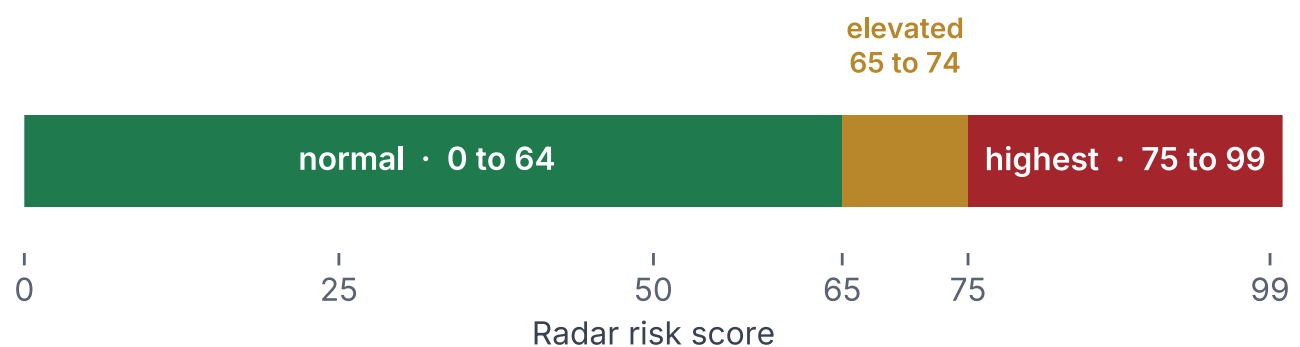


Figure 12.1 Radar risk score bands. By default, payments in the highest band are blocked. The elevated band is where your own rules and review process matter.

LEVEL	SCORE	DEFAULT TREATMENT AND WHAT TO DO
normal	0 to 64	Allowed. The overwhelming majority of legitimate traffic.
elevated	65 to 74	Allowed by default. This is the band worth routing to manual review for high-value orders, and the band where a custom rule earns its keep.
highest	75 to 99	Blocked by default. Overriding this requires a very specific reason and a plan for the fraud you will absorb.
not_assessed	n/a	Radar did not evaluate the payment, typically because it is not a card payment.

Table 12.1 Risk levels. The score is available on the charge's outcome object, so you can record it and correlate it against your own dispute history.

12.2 Card testing, specifically

Card testing deserves its own treatment because it is the attack most likely to hit you, it does not target your product, and its damage is indirect.

The attacker holds a list of card numbers of uncertain validity. They need a merchant who will attempt an authorisation cheaply and report the result. Your checkout is ideal: it is public, it is automated, and a successful small charge confirms a live card.

SYMPTOM	WHAT IT MEANS
Sudden volume of small-value attempts	Automated testing in progress. Often on the smallest purchasable item.
Authorisation rate collapses	Most tested cards are dead. Networks and issuers notice your ratio.
Many distinct cards, few distinct customers or IPs	The signature of a script rather than shoppers.
Attempts on an endpoint that requires no login	The endpoint is the target, not the product.
Network fines or a warning from Stripe	The situation has already been detected upstream of you.

Table 12.2 Card testing indicators. The financial harm is mostly in fees, fines and a damaged authorisation rate rather than in the fraudulent charges themselves.

Defences, in order of effectiveness

1. **Require authentication before a payment can be created.** An attacker who must register and verify an email address per attempt has lost the economics of the attack. This is the single most effective control and it is architectural, not configurable.
2. **Rate limit payment creation** per IP, per account and per device fingerprint. Not per session, which is trivially reset.
3. **Enable Stripe's card testing protections** and the relevant built-in Radar rules.
4. **Add friction on the cheapest item.** Testers gravitate to the lowest amount available. Requiring 3D Secure below a threshold inverts the usual logic and is effective.
5. **Do not expose an unauthenticated endpoint that creates PaymentIntents** with a client-controlled amount. This is the same defect as Chapter 8, now with a second consequence.

TRAP

A public "donate any amount" or "pay what you want" form with no account requirement is a purpose-built card testing oracle. If your product genuinely needs one, put a challenge in front of it, enforce a minimum, rate limit hard by IP and device, and monitor the authorisation rate on that endpoint separately from the rest of the business.

12.3 Writing rules that do not cost you customers

Radar rules let you block, review or allow based on payment attributes. They are powerful and they are easy to misuse, because the cost of a false positive is invisible: the customer simply leaves and you never learn why.

RULE INTENT	ACTION	CAUTION
Block when risk level is highest	Block	Default. Leave it on.
Review high-value orders at elevated risk	Review	Good use of manual review. Requires someone to actually process the queue within hours, not days.
Block when card country differs from IP country	Block	Poor rule. Blocks travellers, expatriates, VPN users and anyone with a foreign card. High false positive rate for modest fraud reduction.
Require 3DS above a value threshold	Request 3DS	Good. Shifts liability precisely where exposure is largest.
Block a specific fingerprint after confirmed fraud	Block	Good, and narrow. Keep a review cadence so the list does not accumulate stale entries.
Block entire countries	Block	Use only with a clear commercial or legal reason. It is a blunt instrument and it is invisible in your conversion metrics.

Table 12.3 Rule patterns. Test every new rule in Radar's simulation against historical traffic before enabling it, so you can see what it would have blocked.

RULE

Every rule you add must have an owner, a stated hypothesis, and a review date. Rules accumulate, their authors leave, and three years later a business is quietly blocking a growing market segment because of a rule nobody remembers writing.

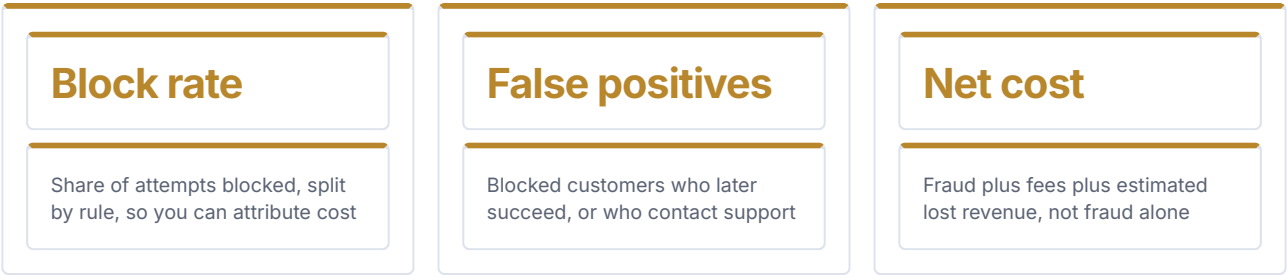
12.4 The signals you can give Radar

Radar performs better when you feed it context that only you have.

- Attach your own customer identifier so repeat, trusted customers are recognisable across payments.
- Collect billing address and postal code when your model allows it; address verification is a meaningful signal.
- Include the Radar session identifier from the client library when using a custom flow, so device signals reach the model.
- Report confirmed fraud you detect yourself, and mark false positives, so the model learns from your outcomes.
- Use consistent metadata for order type and channel, which makes rules expressible in your own business terms rather than in generic payment attributes.

12.5 Measuring the trade-off honestly

The temptation is to optimise the fraud rate alone, because it is the number that appears on a statement. The number that matters is the sum of fraud losses, dispute fees and lost legitimate revenue.



Instrument the blocked path. When a payment is blocked, record it with enough context to recognise later that the same customer succeeded on a second attempt, or contacted support, or never returned. Without that instrumentation you are optimising one side of a two-sided problem and you will over-block indefinitely, because over-blocking looks like success.

—

CHAPTER THIRTEEN

Disputes, Chargebacks and Evidence

A dispute is a customer telling their bank the charge was wrong, and you will lose most of the ones you contest. What decides the outcome is the evidence your system recorded at the time of purchase.

When a cardholder disputes a charge, the issuer withdraws the funds from your account immediately and charges a dispute fee. You then have a limited window to submit evidence. The issuer decides. If you win, the funds return; the fee generally does not. If you lose, you are out the amount, the fee and the product.

The asymmetry is deliberate. Consumer protection regulation places the burden on the merchant, and card networks enforce thresholds: sustained dispute rates above roughly one percent attract monitoring programmes, fines and, eventually, loss of processing.

13.1 What disputes are actually about

REASON	SHARE	WHAT IT USUALLY MEANS
Fraudulent	Largest	The cardholder says they did not authorise it. Sometimes true, and frequently "friendly fraud": the customer did buy it and does not recognise the charge or has changed their mind.
Product not received	Common	Delivery failed, was slow, or the customer could not find the digital product. Often a support failure rather than a fraud one.
Product unacceptable	Common	Received but not as described. A product and expectations problem.
Subscription cancelled	Common	The customer believed they had cancelled. Frequently caused by a cancellation flow that is harder than the signup flow.
Duplicate	Occasional	Charged twice. Usually a real integration bug: a missing idempotency key. Chapter 8.
Credit not processed	Occasional	A refund was promised and not issued, or not issued fast enough.
Unrecognised	Occasional	The billing descriptor does not match anything the customer remembers buying.

Table 13.1 Dispute reasons. Note how many are operational rather than adversarial. Most dispute reduction is achieved by fixing product and support, not by fighting harder.

The cheapest dispute is the refund you issued before the customer thought to call their bank.

13.2 Prevention, ranked by return

1. **A recognisable billing descriptor.** The single highest-return change available. If your legal entity is one name and your product is another, the customer sees an unfamiliar line on a statement and calls their bank. Set the descriptor to the name customers know, include a support contact where the format allows, and use dynamic descriptors for distinct product lines.
2. **A cancellation flow that is at least as easy as signup.** Self-service, one or two clicks, immediate confirmation by email. Retention flows that require contacting support convert directly into disputes, and in several jurisdictions they are now unlawful.
3. **Proactive renewal notices.** Email before an annual renewal charges. A customer who is reminded and does nothing has consented; a customer surprised by a large charge disputes it.
4. **Fast, generous refunds.** A refund costs you the transaction. A dispute costs you the transaction, the fee, the staff time and a point of dispute ratio. Refund first, argue never.
5. **Visible, responsive support.** Many disputes are the customer's second choice after failing to reach you. A reachable support address on the receipt and on the site prevents a measurable share.
6. **3D Secure on higher-value transactions.** Shifts liability for fraud disputes to the issuer. Chapter 5.
7. **Clear delivery expectations and tracking.** For physical goods, tracked delivery with signature above a value threshold is both prevention and evidence.

PRACTICE

Instrument your own dispute reasons and review them monthly with whoever owns the product. A cluster of "subscription cancelled" disputes is a cancellation flow bug. A cluster of "unrecognised" is a descriptor problem. Both are cheap to fix and neither is visible from the payments dashboard alone.

13.3 Responding, when you must

Stripe delivers `charge.dispute.created` and gives you a window to submit evidence, typically shorter than the network's own deadline so that Stripe can forward it in time. Treat the deadline as immovable.

EVIDENCE	STRONGEST FOR	NOTES
Proof of delivery with signature	Product not received	Tracking to the billing address is far stronger than tracking to any address.
Access and usage logs	Fraudulent, digital goods	Timestamps, IP addresses, device, and activity after purchase. Showing the account was used extensively undermines a non-authorisation claim.
Authentication records	Fraudulent	3DS result, login history, email verification. If the payment was authenticated, say so prominently.

EVIDENCE	STRONGEST FOR	NOTES
Signed terms and cancellation policy	Subscription cancelled, unacceptable	Include the timestamp of acceptance and the exact text presented at that time.
Customer correspondence	All types	Support threads showing the issue was raised and addressed, or never raised at all.
Refund policy shown at check-out	Credit not processed	A screenshot of the actual page as displayed to that customer, not the current version.
Billing descriptor evidence	Unrecognised	What appeared on their statement and how it maps to your brand.

Table 13.2 Evidence by dispute type. Compelling evidence is specific, timestamped and machine-generated. Narrative explanations persuade nobody.

RULE

Collect evidence continuously, not at dispute time. If your system does not already record when terms were accepted, which IP created the account, when the product was accessed and what the descriptor was on that date, you cannot reconstruct it two months later. Build the evidence trail into the fulfilment path from Chapter 10.

```
// Capture evidence at the moment it exists, not when you need it.
await db.insert("purchase_evidence", {
  charge_id:      charge.id,
  user_id:        user.id,
  terms_version:  TERMS_VERSION,
  terms_accepted_at: user.termsAcceptedAt,
  signup_ip:      user.signupIp,
  purchase_ip:    req.ip,
  user_agent:     req.headers["user-agent"],
  three_d_secure: charge.payment_method_details?.card?.three_d_secure?.result,
  billing_descriptor: charge.statement_descriptor ?? DEFAULT_DESCRIPTOR,
  delivery_reference: fulfilment.trackingNumber ?? null,
  first_access_at: null, // updated on first product use
});
```

A purpose-built evidence table. It costs one insert per purchase and it is the difference between a defensible response and an apologetic one.

13.4 Handling the events

EVENT	ACTION
<code>charge.dispute.created</code>	Revoke access if your model requires it, freeze the account for review, assemble evidence automatically, and notify a human with a deadline.
<code>charge.dispute.updated</code>	Track state changes and additional information requests.
<code>charge.dispute.closed</code>	Record the outcome. Won means restore access; lost means confirm revocation and record the reason for your monthly review.

EVENT	ACTION
charge.dispute.funds_withdrawn	Reflect in your own accounting immediately. Do not wait for the payout statement.
charge.dispute.funds_reinstated	The reverse.
radar.early_fraud_warning.created	The issuer has flagged the charge as fraudulent before a dispute exists. Refunding proactively at this point usually prevents the dispute entirely and keeps it off your ratio.

Table 13.3 Dispute lifecycle events. The early fraud warning is the most actionable and the most frequently ignored.

PRACTICE

Automate the response to early fraud warnings: refund immediately, revoke access, and flag the customer. The charge is almost certainly fraudulent, the dispute is coming, and refunding first keeps it out of your dispute ratio. This is one of very few places in payments where a fully automatic action is clearly correct.

13.5 Watching the ratio

Networks measure disputes as a proportion of transactions, monitored monthly. Exceeding the threshold moves you into a remediation programme with fines that escalate, and persistent failure ends in losing the ability to process cards.

Below 0.5% Comfortable. Normal operating range for most businesses	0.5% to 0.9% Investigate now. You are one bad month from a programme	Above 1% Network monitoring territory. Treat as an incident
---	---	--

Thresholds vary by network and by programme and they change; confirm the current figures with your acquirer. The engineering response, however, is constant: instrument the reason codes, attribute them to product surfaces, and fix the surfaces. A dispute ratio is a lagging indicator of a product problem that your support inbox already knew about.



PART FIVE

Platforms and Marketplaces

The moment money passes through you on its way to someone else, you have become a different kind of business with a different liability model. Connect is the product for that, and the choice of account type is close to irreversible.

CHAPTER FOURTEEN

Connect: Account Types and Liability

Paying third parties is not a feature you bolt onto an existing integration. It changes your regulatory position, your data model and the list of things that can go wrong at your expense.

Connect exists for businesses where money flows through the platform to someone else: a marketplace paying sellers, a booking service paying venues, a software platform enabling its customers to accept payments. The defining question is not technical. It is: *when a customer disputes a charge, whose money is taken?*

RULE

If funds from your customers are destined for a third party, you must use Connect. Collecting into your own account and paying out by transfer makes you an unlicensed money transmitter in many jurisdictions, and it is an explicit violation of Stripe's terms. This is not a technical shortcut with a compliance footnote; it terminates accounts.

14.1 The properties that actually differ

Connect account types are conventionally named Standard, Express and Custom, and it is more useful to think in terms of the underlying properties, because those are what you are choosing between.

PROPERTY	QUESTION IT ANSWERS	CONSEQUENCE
Loss liability	Who absorbs disputes, refunds and negative balances?	If it is you, you need reserves, underwriting judgement and a way to recover from sellers. This is the single most expensive choice on the page.
Fee payer	Who pays Stripe's processing fees?	Determines how you model your take rate and how transparent pricing is to sellers.
Requirement collection	Who gathers identity and compliance information?	If Stripe does it, onboarding is fast and standard. If you do it, you own an ongoing compliance workflow that changes as regulation changes.
Dashboard access	Does the seller get a Stripe dashboard?	Determines whether sellers self-serve their own reporting and disputes, or whether every question becomes your support ticket.

Table 14.1 The four controller properties. The named account types are presets over these; understanding the properties makes the choice legible.

TYPE	ONBOARDING	LOSS LIABILITY	SELLER DASH-BOARD	BEST FOR
Standard	Stripe, seller has own account	The seller	Full Stripe dashboard	Platforms whose sellers are established businesses that can manage their own payments relationship. Lowest liability and lowest effort for you.
Express	Stripe-hosted, branded to you	Configurable, often the platform	Limited express dashboard	Marketplaces and gig platforms. The common default: fast onboarding, decent seller experience, Stripe handles compliance collection.
Custom	You collect everything	The platform	None, you build it	Platforms that need the seller never to see Stripe. Maximum control, maximum obligation, and substantially more engineering than teams expect.

Table 14.2 Account types compared. Choosing Custom to control the user experience, without appreciating the compliance workload attached, is a recurring and expensive mistake.

TRAP

Choosing Custom for aesthetics. The seller never sees Stripe, which is appealing. In exchange you now own identity verification, ongoing requirement updates as regulation changes, dispute handling on the seller's behalf, negative balance recovery, and a support burden that grows linearly with your seller count. Express achieves most of the branding benefit for a fraction of the obligation.

14.2 Charge types

Independently of account type, there are three ways to structure a charge, and the choice determines who appears on the customer's statement and where liability initially sits.

CHARGE TYPE	MECHANICS	STATEMENT SHOWS	NOTES
Direct	Created on the connected account. Platform takes an application fee.	The seller	Liability sits with the seller. Closest to the seller being the merchant of record. Natural fit with Standard accounts.
Destination	Created on the platform, funds automatically transferred to the connected account.	The platform	The platform is the merchant of record and generally bears loss liability. Most common marketplace pattern.
Separate charges and transfers	Charge on the platform; transfers created independently, possibly to several accounts.	The platform	Necessary when one customer payment splits across multiple sellers, or when the split is not known at charge time. Most flexible, most bookkeeping.

Table 14.3 Charge types. The statement descriptor column matters more than it looks: it drives "unrecognised charge" disputes, per Chapter 13.

14.3 The failure modes specific to platforms

Negative balances on connected accounts

A seller receives a payout, then a customer disputes an earlier charge. The seller's balance goes negative and there is nothing to recover from. Depending on the liability configuration, that shortfall becomes yours. Mitigate with payout delays proportional to your risk, reserves on new sellers, and a graduated release schedule as a seller establishes history.

Onboarding abandonment

Sellers begin onboarding, hit a requirement, and stop. They then appear in your product as active but cannot receive money, and the failure surfaces only when a payout fails. Monitor `account.updated` and surface outstanding requirements in your own interface rather than waiting for the seller to notice.

Requirement changes over time

Compliance requirements are not collected once. Regulation changes, thresholds are crossed, documents expire. A previously complete account can become restricted with no action on the seller's part. This is the ongoing operational cost of Custom accounts that surprises platforms.

Cross-border complications

Sellers in other countries mean currency conversion, differing payout schedules, differing identity requirements and tax reporting obligations that vary by jurisdiction. Each new seller country is a project, not a configuration change.

Application fee refunds

When a charge is refunded, decide explicitly whether your platform fee is refunded too. Both answers are defensible; not deciding means the behaviour is whatever the default was, and your sellers will eventually notice.

Webhook routing

Connect events arrive with an `account` field identifying the connected account. A handler written for the platform that ignores this field will apply a seller's event to the platform's own state. Route explicitly on `event.account`, and use separate endpoints where practical.

```

export async function POST(req: Request) {
  const raw = await req.text();
  const event = stripe.webhooks.constructEvent(
    raw, req.headers.get("stripe-signature")!, CONNECT_WEBHOOK_SECRET);

  // Connect events carry the connected account they belong to.
  const accountId = event.account;
  if (!accountId) {
    // A platform event arrived on the Connect endpoint. Do not guess.
    logger.error({ eventId: event.id }, "platform event on connect endpoint");
    return new Response(null, { status: 200 });
  }

  const seller = await getSellerByStripeAccount(accountId);
  if (!seller) {
    logger.warn({ accountId }, "event for unknown connected account");
    return new Response(null, { status: 200 });
  }

  await enqueue({ eventId: event.id, sellerId: seller.id });
  return new Response(null, { status: 200 });
}

```

Explicit account resolution. The two early returns are guards against the most common Connect handler bug: applying a seller's event to the platform.

14.4 Deciding

IF THIS IS TRUE	THEN
Your sellers are established businesses who already accept payments	Standard. Let them own the relationship and the liability.
Your sellers are individuals or small operators, and onboarding friction is a growth constraint	Express. The default answer for most marketplaces.
Payments must be entirely invisible inside your product, and you have compliance capacity	Custom, with clear eyes about the ongoing obligation.
One customer payment splits across several sellers	Separate charges and transfers, regardless of account type.
You are not sure yet	Express with destination charges. It is the most reversible position and it covers the majority of cases.

Table 14.4 A decision table for Connect. Revisit it before you have sellers, because migrating account types after the fact requires re-onboarding every one of them.

PART SIX

Operating the Integration

Payments are not shipped once. Versions move, environments diverge, and the code that was correct at launch drifts. This part is about keeping it correct, and about the two checklists you will actually use.

- 15 API Versioning and SDK Discipline
- 16 Environments, Sandboxes and Testing
- 17 The Go-Live Checklist
- 18 The Security Review Grid

CHAPTER FIFTEEN

API Versioning and SDK Discipline

Stripe ships changes continuously and pins your account to a version so they do not break you. That protection ends the moment someone upgrades a library without reading the changelog.

15.1 How versioning works

Every Stripe account has a default API version, set when the account was created and changed only deliberately. Requests use that version unless they override it. Versions are dated and grouped into named release trains, so a version string looks like a date followed by a name.

MECHANISM	WHERE IT IS SET	BEHAVIOUR
Account default version	Stripe dashboard	Applies to every request that does not override it, and to the shape of webhook event payloads.
Per-request override	<code>Stripe-Version</code> header, or the SDK constructor	Lets one service move ahead of the account default. Useful for staged migration.
Webhook endpoint version	Per endpoint configuration	Each endpoint can receive events in a specific version, which allows a new handler to be developed against a newer shape while the old one keeps running.
SDK major version	Your dependency manifest	Library major versions are pinned to API versions. Upgrading the SDK can change the API version your code speaks, whether or not you intended it.

Table 15.1 Version control points. The last row is where teams get caught, because a routine dependency bump does not look like a payments change.

RULE

Pin the API version explicitly in code, not implicitly through the SDK. An explicit `apiVersion` in the client constructor means a library upgrade cannot silently change the wire format your handlers parse.

```
import Stripe from "stripe";

export const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: "2026-02-25.clover", // explicit, reviewed, changed deliberately
  appInfo: { name: "acme-billing", version: RELEASE_SHA },
  timeout: 20_000,
  maxNetworkRetries: 2,
});
```

Confirm the version string you pin against Stripe's current changelog. The point is that it is a decision recorded in your repository, not a side effect of a lockfile.

15.2 Upgrading safely

1. Read the changelog for every version between your current one and the target. Breaking changes are documented individually and they are not always in areas you expect.
2. Create a second webhook endpoint on the new version, pointed at a new handler, and run both in parallel. Compare the outcomes.
3. Override the version per request in one non-critical service first, and observe.
4. Move the account default only after every consumer has been verified against the new shape.
5. Keep the old endpoint until you have seen a full billing cycle on the new one, including monthly and annual renewals.

TRAP

Upgrading the account default version on a Friday because a dependency audit flagged the SDK. Webhook payload shapes change with the account version, so every handler you own begins receiving a different structure simultaneously, including handlers in services nobody has touched in a year. Version changes are billing changes and deserve the same care as a schema migration.

15.3 Defensive parsing

Even between versions, payload shapes evolve: fields are added, optional fields appear, and expanded objects can arrive as either an identifier string or a full object depending on how the request was made.

```
// A field that may be a string ID or an expanded object.
function customerId(input: string | Stripe.Customer | Stripe.DeletedCustomer | null) {
  if (!input) return null;
  return typeof input === "string" ? input : input.id;
}

// Validate the shape you actually depend on, rather than trusting it.
const Session = z.object({
  id: z.string(),
  status: z.enum(["open", "complete", "expired"]),
  payment_status: z.enum(["paid", "unpaid", "no_payment_required"]),
  client_reference_id: z.string().nullable(),
  metadata: z.record(z.string()).default({}),
});

const parsed = Session.safeParse(session);
if (!parsed.success) {
  logger.error({ sessionId: session.id }, "unexpected session shape");
  throw new Error("Schema drift");
}
```

Schema validation at the boundary turns a silent misparse into a loud, attributable failure. On a payment path, loud is what you want.

15.4 Keeping the SDK current

- Track SDK releases deliberately, and read release notes rather than auto-merging.

- Separate the SDK upgrade commit from the API version change commit, so each can be reverted independently.
 - Run the full payment test suite, including the security assertions from Chapter 9, on every SDK change.
 - Record the SDK and API version in `appInfo`, so Stripe's support and your own logs can correlate behaviour with a specific build.
 - Do not run an SDK version so old that it no longer receives security fixes. Staying current is a security control, not only a feature one.
-

CHAPTER SIXTEEN

Environments, Sandboxes and Testing

Most payment bugs reach production because the tests only ever covered the path where everything works. This chapter is about testing the rest.

16.1 Separating environments properly

ENVIRONMENT	KEYS	RULES
Local development	Test mode only	Every developer uses test keys. Webhooks forwarded through the CLI. No live key ever reaches a laptop.
Continuous integration	Test mode only	A dedicated test key, rotatable independently, so a compromised CI system cannot touch live data.
Staging	Test mode, separate sandbox	Its own sandbox where possible, so staging data does not pollute the data developers are debugging against.
Production	Live mode	Live keys exist only here, only in the secret manager, accessible to a small named group.

Table 16.1 Environment separation. Test and live mode data are completely isolated in Stripe: identifiers do not cross, and a test object cannot be referenced from live mode.

TRAP

A single shared test account across every environment and every developer. Webhook events fan out to whoever happens to be listening, one developer's test data breaks another's assumptions, and CI runs become non-deterministic because a colleague cancelled a subscription mid-run. Use separate sandboxes where the platform supports them, and namespace test data with metadata that identifies its origin.

16.2 What to test, beyond the happy path

The happy path is the least valuable test in the suite, because it is the one path that is manually exercised constantly. The valuable tests are the ones nobody performs by hand.

CATEGORY	CASES
Declines	Generic decline, insufficient funds, lost card, stolen card, expired card, incorrect CVC, processing error. Each should surface a distinct, human message and none should grant entitlement.
Authentication	3DS required and completed. 3DS required and abandoned. 3DS required and failed. Off-session charge returning <code>authentication_required</code> .
Asynchronous methods	A session that completes with <code>payment_status: unpaid</code> , then succeeds later. The same, but failing later. This is where Table 10.1 becomes concrete.

CATEGORY	CASES
Webhook security	The seven assertions listed at the end of Chapter 9. These are the highest value tests in the suite.
Idempotency	Same event twice. Ten concurrent deliveries. A retried outbound call with a fixed key.
Ordering	Events applied out of chronological order. The stale event must be discarded, not applied.
Revocation	Refund, dispute, cancellation, downgrade, failed renewal. Each must remove exactly the right access.
Price integrity	Attempt to create a session with a modified amount, a foreign price identifier, a negative quantity, and a plan key not in the catalogue. All four must be rejected.
Identity binding	Attempt to fulfil another user's session ID. Attempt to pass a different <code>userId</code> in the body. Both must fail.
Failure injection	Stripe times out. The database is unavailable mid-fulfilment. The queue is down. Each must leave the system in a recoverable state.

Table 16.2 The test matrix. The last four categories are security tests as much as functional ones, and they are the ones that would fail on most integrations I have reviewed.

16.3 Testing tools

```
# Forward live test-mode events to a local endpoint
stripe listen --forward-to localhost:3000/api/webhooks/stripe

# Trigger specific events on demand
stripe trigger payment_intent.succeeded
stripe trigger invoice.payment_failed
stripe trigger charge.dispute.created
stripe trigger customer.subscription.updated

# Inspect what was actually sent
stripe events list --limit 10
stripe events resend evt_1234567890

# Tail API requests in real time while reproducing a bug
stripe logs tail
```

PRACTICE

Write an integration test that constructs a webhook request and signs it with your test signing secret, so the entire verification path is exercised in CI rather than mocked. Mocking `constructEvent` is the most common way a suite ends up with excellent coverage of a handler whose security control has never been executed once.

```
import { createHmac } from "node:crypto";

function signedRequest(payload: object, secret: string, timestamp = Date.now() / 1000) {
  const body = JSON.stringify(payload);
  const t = Math.floor(timestamp);
  const signature = createHmac("sha256", secret)
    .update(`${t}.${body}`)
    .digest("hex");
  return {
    body,
    headers: { "stripe-signature": `t=${t},v1=${signature}` },
  };
}

test("rejects an event signed with the wrong secret", async () => {
  const { body, headers } = signedRequest(sampleEvent, "whsec_wrong");
  const res = await POST(new Request(URL, { method: "POST", body, headers }));
  expect(res.status).toBe(400);
  expect(await countEntitlements()).toBe(0);
});

test("rejects an event with a stale timestamp", async () => {
  const hourAgo = Date.now() / 1000 - 3600;
  const { body, headers } = signedRequest(sampleEvent, REAL_SECRET, hourAgo);
  const res = await POST(new Request(URL, { method: "POST", body, headers }));
  expect(res.status).toBe(400);
});

test("grants exactly one entitlement for ten concurrent deliveries", async () => {
  const { body, headers } = signedRequest(sampleEvent, REAL_SECRET);
  await Promise.all(Array.from({ length: 10 }, () =>
    POST(new Request(URL, { method: "POST", body, headers })))));
  await drainQueue();
  expect(await countEntitlements()).toBe(1);
});
```

Three tests that between them cover forgery, replay and concurrency. If your suite has only these three additions, it is materially safer than most.

16.4 Before the first live payment

There is one test that cannot be performed in test mode, and it is the most important one: make a real payment, with a real card, for a real amount, and follow it all the way through. Confirm the charge appears, the webhook fires, fulfilment runs, the receipt arrives, the descriptor on the statement is what you expected, and then refund it and confirm revocation works. Do this before launch, not after the first customer.

CHAPTER SEVENTEEN

The Go-Live Checklist

Everything that has to be true before real money moves. Run it as a list with a named owner on every line, not as a shared feeling that things are probably fine.

17.1 Account and business readiness

- ☐ Account activated, with every outstanding verification requirement cleared.
- ☐ Bank account added, verified, and confirmed to be in the legal entity's name.
- ☐ Business category reviewed against the restricted list, with written confirmation for anything near a line.
- ☐ Public statement descriptor set to the name customers actually recognise, and checked on a real statement.
- ☐ Support email and phone number configured on the account and visible on the site.
- ☐ Terms of service, refund policy and privacy policy published and linked from checkout.
- ☐ Tax configuration reviewed, including whether automatic tax applies to your model.
- ☐ Payout schedule understood, including the first-payout delay for new accounts.
- ☐ Team members invited with least-privilege dashboard roles, not shared logins.
- ☐ Multi-factor authentication enforced for every user with dashboard access.

17.2 Keys and secrets

- ☐ Live keys exist only in the production secret manager.
- ☐ No secret appears in any repository, in any branch, in any historical commit.
- ☐ Secret scanning enabled in CI and as a pre-commit hook.
- ☐ The build guard from section 7.4 is wired into the prebuild step and verified to fail.
- ☐ Restricted keys issued for every component that does not need full account authority.
- ☐ The webhook signing secret is distinct per endpoint and stored alongside the other secrets.
- ☐ Key rotation rehearsed at least once, in test mode, end to end.
- ☐ The leaked key runbook from section 7.6 is written down and its location is known.

17.3 The integration itself

- ☐ Prices resolved from a server-owned catalogue; no `price_data` built from request input anywhere in the codebase.
- ☐ User identity taken from the authenticated session in every payment-creating handler.

- ☐ Idempotency keys present on every outbound write, and derived from intent rather than randomly generated.
- ☐ Webhook signature verification present, using the raw body, with no code path that bypasses it.
- ☐ Event deduplication implemented with an atomic insert, and tested under concurrency.
- ☐ The webhook handler returns 200 promptly and performs work asynchronously.
- ☐ Fulfilment is a single idempotent function called from both the webhook and the return page.
- ☐ The return page verifies session ownership before doing anything.
- ☐ Both `status` and `payment_status` are checked before fulfilling.
- ☐ Subscription state mirrored locally, with an out-of-order guard on updates.
- ☐ Every grant path has a matching, tested revoke path.
- ☐ API version pinned explicitly in the client constructor.
- ☐ Timeouts and bounded retries configured on the Stripe client.

17.4 The payment page

- ☐ HTTPS everywhere, with HSTS, and no mixed content on any checkout route.
- ☐ Content Security Policy enforced, with no `unsafe-inline` in `script-src`.
- ☐ CSP violation reports collected and alerting.
- ☐ Written inventory of every script on the payment page, with an owner and a justification.
- ☐ No tag manager, analytics, session replay or chat widget on the checkout route.
- ☐ Automated weekly comparison of payment page content and headers against a known-good manifest.
- ☐ No subresource integrity hash pinned to Stripe's evergreen loader.

17.5 Fraud, disputes and monitoring

- ☐ Radar reviewed; default block on the highest risk band left enabled.
- ☐ Card testing protections enabled, and payment creation rate limited.
- ☐ Every custom rule has a named owner and a review date.
- ☐ Early fraud warnings handled, ideally with an automatic refund.
- ☐ Dispute evidence captured at purchase time, not assembled retrospectively.
- ☐ Alerting configured for: webhook delivery failure, unprocessed events, payments without fulfilment, fulfilments without payment, reconciliation drift, authorisation rate drop, dispute rate.
- ☐ Nightly reconciliation job running and reporting zero drift.
- ☐ Someone is on call for billing, and they know these alerts exist.

17.6 The final rehearsal

- ☐ A real payment made with a real card, followed end to end through fulfilment.
- ☐ The statement descriptor confirmed on an actual bank statement.
- ☐ That payment refunded, and revocation confirmed to have worked.
- ☐ A dispute simulated in test mode and the evidence assembly path exercised.
- ☐ The webhook endpoint deliberately broken in staging, to confirm alerting fires.
- ☐ Rollback plan written for the payment path specifically.

RULE

Any unchecked box in sections 17.2 and 17.3 blocks launch. The others can be scheduled. That distinction is worth agreeing on before launch pressure makes everything feel negotiable.

CHAPTER EIGHTEEN

The Security Review Grid

Seventy-two checks for auditing an integration you did not write, ordered so that the first twenty findings are usually the ones that matter.

This is the working instrument of the book. When I am handed a repository and asked whether it is safe to put behind real money, this is what I run. Each row is a question with a verifiable answer, a severity, and a chapter reference for the reasoning.

Severities: **CRITICAL** means money or data is at direct risk today. **HIGH** means a realistic path to loss exists. **MEDIUM** means correctness or compliance is affected. **LOW** means hygiene and maintainability.

A Secrets and access (10 checks)

ID	CHECK	SEVERITY	REF
SEC-01	No <code>sk_</code> , <code>rk_</code> or <code>whsec_</code> value appears in any tracked file, in any branch, or in git history.	CRITICAL	7.4
SEC-02	No secret is assigned to a variable with a client-exposed prefix such as <code>NEXT_PUBLIC_</code> .	CRITICAL	7.4
SEC-03	The production bundle, searched for <code>sk_live</code> and <code>rk_live</code> , contains neither.	CRITICAL	7.4
SEC-04	A database service role or superuser key is never reachable from client code.	CRITICAL	7.8
SEC-05	Row level security is enabled on every table holding customer, subscription or entitlement data, and verified by querying with the public key.	CRITICAL	7.8
SEC-06	Restricted keys are used for components that do not need full account authority.	HIGH	7.3
SEC-07	Secret scanning runs in CI and as a pre-commit hook.	HIGH	7.7
SEC-08	Multi-factor authentication is enforced on every Stripe dashboard account.	HIGH	17.1
SEC-09	The registered webhook endpoint list contains only endpoints you recognise.	HIGH	7.6
SEC-10	A written key rotation procedure exists and has been rehearsed.	MEDIUM	7.5

B The payment creation path (10 checks)

ID	CHECK	SEVERITY	REF
PAY-01	No handler constructs <code>price_data</code> , <code>unit_amount</code> or <code>amount</code> from request input.	CRITICAL	8.1
PAY-02	Price identifiers are selected from a server-owned catalogue, never passed through from the client.	CRITICAL	8.2
PAY-03	User identity is derived from the authenticated session, never from a request body field.	CRITICAL	8.2
PAY-04	Payment creation endpoints require authentication.	HIGH	12.2
PAY-05	Quantity is validated as a bounded positive integer, and rejected rather than clamped.	HIGH	8.3
PAY-06	Discounts and promotion codes are resolved server side or delegated entirely to Stripe.	HIGH	8.4
PAY-07	Idempotency keys are present on outbound writes and derived from intent, not randomly generated per attempt.	HIGH	8.5
PAY-08	Amounts are handled as integers in minor units, with zero-decimal currencies accounted for.	MEDIUM	8.3
PAY-09	Client secrets are never placed in URLs, logs or persistent storage.	MEDIUM	8.7
PAY-10	Metadata contains your own primary key and no sensitive data.	LOW	8.6

C Webhooks (12 checks)

ID	CHECK	SEVERITY	REF
WHK-01	Signature verification is present on every webhook endpoint.	CRITICAL	9.2
WHK-02	Verification uses the raw, unparsed request body.	CRITICAL	9.3
WHK-03	No code path reaches business logic without passing verification, including error branches and feature flags.	CRITICAL	9.2
WHK-04	Verification is not mocked or disabled in any environment that is reachable from the internet.	CRITICAL	16.1
WHK-05	Event deduplication uses an atomic insert on the event ID, not a check-then-act sequence.	CRITICAL	9.5
WHK-06	The timestamp tolerance has not been widened beyond the library default.	HIGH	9.2
WHK-07	The signing secret is read from configuration, not hard-coded.	HIGH	7.1
WHK-08	The handler returns 2xx promptly and performs work asynchronously.	HIGH	9.6

ID	CHECK	SEVERITY	REF
WHK-09	Consequential handlers re-read the object from the API rather than trusting the payload snapshot.	HIGH	9.4
WHK-10	Unhandled event types return 200 and change nothing.	MEDIUM	9.4
WHK-11	Connect handlers resolve and validate <code>event.account</code> before acting.	HIGH	14.3
WHK-12	Event payloads are not written to application logs.	MEDIUM	9.7

D Fulfilment and state (10 checks)

ID	CHECK	SEVERITY	REF
FUL-01	Fulfilment is never triggered solely by the customer reaching the success page.	CRITICAL	10.1
FUL-02	The return page verifies that the session belongs to the requesting user.	CRITICAL	10.2
FUL-03	Both <code>status</code> and <code>payment_status</code> are checked before value is granted.	CRITICAL	10.3
FUL-04	Fulfilment is idempotent, guarded by an atomic claim, and safe under concurrency.	CRITICAL	10.2
FUL-05	Every grant path has a corresponding revoke path with a test.	HIGH	10.6
FUL-06	Refunds and disputes revoke the entitlement they paid for.	HIGH	10.6
FUL-07	Subscription state is mirrored locally rather than queried per request or set once and forgotten.	HIGH	10.4
FUL-08	Updates are guarded against out-of-order event application.	HIGH	10.4
FUL-09	Entitlement is derived from status in one place, not by scattered status comparisons.	MEDIUM	10.4
FUL-10	A reconciliation job runs on a schedule and alerts on drift.	HIGH	10.5

E Client and payment page (8 checks)

ID	CHECK	SEVERITY	REF
CLI-01	Card data never enters your own DOM or your own server.	CRITICAL	3.2
CLI-02	Every route in the payment flow is HTTPS with HSTS, and free of mixed content.	HIGH	6.2
CLI-03	A Content Security Policy is enforced, without <code>unsafe-inline</code> in <code>script-src</code> .	HIGH	6.3
CLI-04	A written script inventory exists for the payment page, satisfying PCI 6.4.3.	HIGH	4.4

ID	CHECK	SEVERITY	REF
CLI-05	Change and tamper detection covers the payment page content and headers, satisfying PCI 11.6.1.	HIGH	6.5
CLI-06	No tag manager, session replay, chat widget or advertising pixel loads on the checkout route.	MEDIUM	6.6
CLI-07	Subresource integrity is applied to pinned third party scripts and deliberately not to Stripe's loader.	MEDIUM	6.4
CLI-08	<code>frame-ancestors</code> is set, and the security headers in Table 6.2 are present.	MEDIUM	6.2

F Fraud and disputes (7 checks)

ID	CHECK	SEVERITY	REF
FRD-01	No unauthenticated endpoint creates a payment with a client-controlled amount.	CRITICAL	12.2
FRD-02	Payment creation is rate limited per IP, account and device.	HIGH	12.2
FRD-03	Radar's default block on the highest risk band is enabled.	HIGH	12.1
FRD-04	Early fraud warnings are handled, ideally with an automatic refund.	HIGH	13.4
FRD-05	Dispute evidence is captured at purchase time and stored durably.	HIGH	13.3
FRD-06	Every custom Radar rule has an owner, a hypothesis and a review date.	MEDIUM	12.3
FRD-07	The statement descriptor is recognisable, and cancellation is self-service.	MEDIUM	13.2

G Resilience and operations (8 checks)

ID	CHECK	SEVERITY	REF
OPS-01	Connection errors are retried with the same idempotency key, never a fresh one.	CRITICAL	11.2
OPS-02	Card declines and invalid requests are not retried.	HIGH	11.2
OPS-03	Retries use exponential backoff with jitter and a bounded attempt count.	MEDIUM	11.3
OPS-04	Explicit timeouts are configured on the Stripe client.	MEDIUM	11.4
OPS-05	Entitlement checks are served from the local mirror and survive a Stripe outage.	HIGH	11.4
OPS-06	Alerting exists for payments without fulfilment and fulfilments without payment.	HIGH	11.5

ID	CHECK	SEVERITY	REF
OPS-07	Webhook delivery failures and stuck event queues are alerted on.	HIGH	11.5
OPS-08	Bulk jobs use a separate restricted key and avoid peak traffic.	LOW	11.1

H Compliance and governance (7 checks)

ID	CHECK	SEVERITY	REF
60V-01	No CVV, track data or PIN is stored anywhere, including logs and error trackers.	CRITICAL	4.3
60V-02	Request bodies containing payment fields are scrubbed at the source before observability capture.	HIGH	4.3
60V-03	The correct SAQ has been identified and attested within the last twelve months.	HIGH	4.1
60V-04	Saved payment methods were set up with future usage declared on-session.	HIGH	5.3
60V-05	<code>authentication_required</code> is handled by returning the customer on-session, not by retrying.	HIGH	5.3
60V-06	The API version is pinned explicitly in code.	MEDIUM	15.1
60V-07	The business category is permitted, and prior approval obtained where required.	MEDIUM	2.2

Using the grid

A practical sequence for a first pass on an unfamiliar codebase, in roughly the order that finds the most in the least time:

1. Grep the repository and the built bundle for key prefixes. SEC-01 through SEC-03 take five minutes and are the most common critical findings.
2. Find every webhook handler and read the first ten lines of each. WHK-01 through WHK-05.
3. Find every call that creates a Checkout Session or PaymentIntent and trace where the amount comes from. PAY-01 through PAY-03.
4. Find the success or return route and see whether it grants anything. FUL-01 and FUL-02.
5. Check whether entitlement is ever revoked. FUL-05 and FUL-06.
6. Load the checkout page and read the response headers and the network tab. CLI-02 through CLI-06.
7. Then work through the remainder in order.

In practice, steps one to five find the critical issues in about ninety minutes. Everything after that is thoroughness, and thoroughness is what the client is paying for.

APPENDIX A

Test Cards and Simulated Outcomes

A working reference for test mode. Any future expiry date and any three digit CVC will do, four digits for American Express.

CONTEXT

Test cards only function with test mode keys. Stripe maintains the authoritative list and adds to it; verify anything unfamiliar against the current documentation before relying on it in an automated suite.

Successful payments

NUMBER	BRAND	BEHAVIOUR
4242 4242 4242 4242	Visa	The default success card. Use it everywhere a payment should simply work.
4000 0566 5566 5556	Visa, debit	Succeeds, and is reported as a debit card, which matters if your logic branches on funding type.
5555 5555 5555 4444	Mastercard	Succeeds.
2223 0031 2200 3222	Mastercard, 2-series	Succeeds. Worth including, because some naive BIN validators reject 2-series ranges.
5200 8282 8282 8210	Mastercard, debit	Succeeds.
3782 822463 10005	American Express	Succeeds. Fifteen digits, four digit security code.
6011 1111 1111 1117	Discover	Succeeds.

Declines

NUMBER	DECLINE CODE	USE IT TO TEST
4000 0000 0000 0002	generic_decline	The baseline decline path and your customer-facing message.
4000 0000 0000 9995	insufficient_funds	A decline the customer can act on. The message should differ from the generic one.
4000 0000 0000 9987	lost_card	A decline you should not encourage retrying.
4000 0000 0000 9979	stolen_card	Same, and worth flagging internally.
4000 0000 0000 0069	expired_card	Dunning flows and card update prompts.

NUMBER	DECLINE CODE	USE IT TO TEST
4000 0000 0000 0127	incorrect_cvc	Field-level error presentation.
4000 0000 0000 0119	processing_error	A transient failure. This one is reasonable to invite a retry on.
4242 4242 4242 4241	incorrect_number	Client-side validation before the request is even made.

Authentication and 3D Secure

NUMBER	BEHAVIOUR
4000 0025 0000 3155	Requires authentication for the initial payment. Subsequent off-session charges succeed.
4000 0027 6000 3184	Requires authentication on every transaction, including off-session, which is the case that breaks naive renewal logic.
4000 0000 0000 3220	3D Secure 2 authentication required, with the challenge flow.
4000 0084 0000 1629	Authentication succeeds, then the payment is declined for insufficient funds. Excellent for testing that you do not grant access on authentication alone.
4000 0082 6000 3178	Authentication fails, so the payment does not complete.

Fraud and risk

NUMBER	BEHAVIOUR
4100 0000 0000 0019	Always blocked by Radar as the highest risk level. Use it to confirm blocked payments never reach fulfilment.
4000 0000 0000 9235	Elevated risk. Use it to exercise your manual review path, if you have one.
4000 0000 0000 0259	Succeeds, then triggers a dispute marked as fraudulent. The fastest way to test your dispute handling end to end.
4000 0000 0000 2685	Succeeds, then triggers an early fraud warning. Use it to verify automatic refunding.

Triggering events from the CLI

```
stripe trigger payment_intent.succeeded
stripe trigger payment_intent.payment_failed
stripe trigger checkout.session.completed
stripe trigger checkout.session.async_payment_succeeded
stripe trigger checkout.session.async_payment_failed
stripe trigger customer.subscription.created
stripe trigger customer.subscription.updated
stripe trigger customer.subscription.deleted
stripe trigger invoice.paid
stripe trigger invoice.payment_failed
stripe trigger charge.refunded
stripe trigger charge.dispute.created
```

APPENDIX B

Content Security Policy Directives

The origins Stripe's client libraries need, and a complete starting policy for a checkout page.

RULE

Stripe adds origins as it ships payment methods and features. Deploy in report-only mode, observe for a full business cycle, and check the current documentation before enforcing.

Origins by purpose

DIRECTIVE	ORIGIN	NEEDED FOR
<code>script-src</code>	<code>https://js.stripe.com</code>	Stripe.js itself. Required by every browser integration.
<code>frame-src</code>	<code>https://js.stripe.com</code>	The iframes hosting Elements and the card fields.
<code>frame-src</code>	<code>https://hooks.stripe.com</code>	3D Secure challenge frames and redirect-based payment methods.
<code>frame-src</code>	<code>https://checkout.stripe.com</code>	Embedded Checkout.
<code>connect-src</code>	<code>https://api.stripe.com</code>	Tokenisation and confirmation requests from the browser.
<code>connect-src</code>	<code>https://maps.googleapis.com</code>	Address autocomplete, if you enable it in Checkout or the Address Element.
<code>img-src</code>	<code>https://*.stripe.com</code>	Payment method logos and brand marks.
<code>img-src</code>	<code>data:</code>	Inline images used by some Elements.
<code>font-src</code>	<code>https://js.stripe.com</code>	Fonts loaded inside Element iframes when you customise typography.

A complete policy for a checkout page

```
Content-Security-Policy:
  default-src 'none';
  script-src 'self' 'nonce-{PER_REQUEST_NONCE}' https://js.stripe.com;
  style-src 'self' 'nonce-{PER_REQUEST_NONCE}';
  img-src 'self' data: https://*.stripe.com;
  font-src 'self' https://js.stripe.com;
  connect-src 'self' https://api.stripe.com https://maps.googleapis.com;
  frame-src https://js.stripe.com https://hooks.stripe.com https://checkout.stripe.com;
  form-action 'self';
  frame-ancestors 'none';
  base-uri 'self';
  object-src 'none';
  upgrade-insecure-requests;
  report-uri /api/csp-report;
  report-to csp-endpoint
```

Starting from `default-src 'none'` means every capability is explicitly granted. It is more work initially and far easier to reason about afterwards.

Collecting reports

```
// POST /api/csp-report
export async function POST(req: Request) {
  const report = await req.json().catch(() => null);
  if (!report) return new Response(null, { status: 204 });

  const violation = report["csp-report"] ?? report;
  logger.warn({
    documentUri: violation["document-uri"],
    blockedUri: violation["blocked-uri"],
    violatedDirective: violation["violated-directive"],
  }, "csp violation");

  // A violation originating from a payment route is a security event,
  // not a formatting problem. Escalate it.
  if (String(violation["document-uri"] ?? "").includes("/checkout")) {
    await alertSecurityTeam(violation);
  }
  return new Response(null, { status: 204 });
}
```

This endpoint is also part of your 11.6.1 tamper detection story, because an unexpected script on the payment page produces a violation here.

The rest of the header set

```
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=(), payment=(self)
Cross-Origin-Opener-Policy: same-origin-allow-popups
Cache-Control: no-store
```

Note `payment=(self)` rather than an empty allow-list: the Payment Request API needs it if you offer wallet payments.



APPENDIX C

Webhook Events That Matter

Subscribe to what you handle and nothing more. This is the working set for a typical subscription and one-off commerce business.

Checkout

EVENT	PRIORITY	HANDLE BY
<code>checkout.session.completed</code>	Essential	Calling the idempotent fulfilment function. Check <code>payment_status</code> before granting.
<code>checkout.session.async_payment_succeeded</code>	Essential	The same fulfilment function. This is how delayed methods eventually confirm.
<code>checkout.session.async_payment_failed</code>	Essential	Recording the failure and notifying the customer. Do not grant.
<code>checkout.session.expired</code>	Useful	Cleaning up abandoned carts and triggering recovery email.

Payments

EVENT	PRIORITY	HANDLE BY
<code>payment_intent.succeeded</code>	Essential	Fulfilling, if you drive payments directly rather than through Checkout.
<code>payment_intent.payment_failed</code>	Essential	Recording the failure reason and surfacing an actionable message.
<code>payment_intent.requires_action</code>	Useful	Tracking authentication drop-off.
<code>charge.refunded</code>	Essential	Revoking the entitlement the charge paid for.
<code>charge.succeeded</code>	Optional	Analytics. Usually redundant with the intent event.

Subscriptions and invoices

EVENT	PRIORITY	HANDLE BY
<code>customer.subscription.created</code>	Essential	Creating the local mirror row.
<code>customer.subscription.updated</code>	Essential	Applying status, plan and period changes, with the out-of-order guard.
<code>customer.subscription.deleted</code>	Essential	Revoking at the correct boundary.

EVENT	PRIORITY	HANDLE BY
<code>customer.subscription.trial_will_end</code>	Useful	Reminding the customer before the trial converts. Reduces disputes.
<code>customer.subscription.paused</code>	Useful	Suspending access without deleting state.
<code>invoice.paid</code>	Essential	Extending the access period and recording the payment.
<code>invoice.payment_failed</code>	Essential	Starting dunning. Do not revoke on the first failure by default.
<code>invoice.payment_action_required</code>	Essential	Emailing the customer a link to authenticate. Chapter 5.
<code>invoice.upcoming</code>	Useful	Advance notice before a renewal, especially annual ones.

Disputes and risk

EVENT	PRIORITY	HANDLE BY
<code>charge.dispute.created</code>	Essential	Revoking, freezing, assembling evidence, alerting a human with the deadline.
<code>charge.dispute.closed</code>	Essential	Recording the outcome and restoring access if won.
<code>charge.dispute.funds_withdrawn</code>	Useful	Reflecting the withdrawal in your own accounting immediately.
<code>radar.early_fraud_warning.created</code>	Essential	Refunding proactively, before the dispute exists.
<code>review.opened</code> / <code>review.closed</code>	Useful	Driving a manual review queue.

Account and payouts

EVENT	PRIORITY	HANDLE BY
<code>account.updated</code>	Essential	For platforms: surfacing outstanding requirements on connected accounts before payouts fail. For your own account: noticing verification requests.
<code>payout.failed</code>	Essential	Alerting immediately. A failed payout usually means a bank detail problem that will recur.
<code>payout.paid</code>	Useful	Reconciling against your ledger.
<code>customer.updated</code>	Optional	Keeping contact details in sync, if Stripe is authoritative for them.

PRACTICE

Split these across at least two endpoints: one for billing and commerce, one for account and payout events. They have different urgency, different owners and different failure modes, and separating them means a bug in one does not stall the other.



APPENDIX D

Reference Implementation

A complete, minimal, production-shaped integration. TypeScript, Next.js App Router, PostgreSQL. Every principle from Parts Three and Four in one place.

D.1 Schema

```

-- Deduplication for inbound events (Chapter 9).
create table stripe_events (
  id          text primary key,
  type        text          not null,
  received_at timestampz not null default now(),
  processed_at timestampz,
  attempts    int           not null default 0,
  last_error  text
);

-- Idempotency barrier for fulfilment (Chapter 10).
create table fulfilments (
  id                bigserial primary key,
  checkout_session_id text unique not null,
  user_id           uuid not null references users(id),
  status            text not null default 'in_progress',
  completed_at      timestampz,
  last_error        text,
  created_at        timestampz not null default now()
);

-- Local mirror of billing state (Chapter 10).
create table subscriptions (
  id                text primary key,
  user_id           uuid not null references users(id),
  stripe_customer_id text not null,
  status            text not null,
  price_id          text not null,
  current_period_end timestampz not null,
  cancel_at_period_end boolean not null default false,
  last_event_created_at timestampz not null,
  updated_at        timestampz not null default now()
);
create unique index subscriptions_one_active_per_user
  on subscriptions (user_id) where status in ('active', 'trialing');

-- Dispute evidence, captured at purchase time (Chapter 13).
create table purchase_evidence (
  charge_id        text primary key,
  user_id          uuid not null references users(id),
  terms_version     text not null,
  terms_accepted_at timestampz not null,
  purchase_ip       inet,
  user_agent        text,
  three_d_secure    text,
  billing_descriptor text not null,
  first_access_at   timestampz,
  created_at        timestampz not null default now()
);

-- Enable row level security everywhere (Chapter 7).
alter table subscriptions      enable row level security;
alter table fulfilments        enable row level security;
alter table purchase_evidence  enable row level security;

create policy own_subscriptions on subscriptions
  for select using (user_id = auth.uid());
-- Deliberately no insert or update policy: only the server writes here.

```

D.2 The Stripe client

```
// lib/stripe.ts
import Stripe from "stripe";

if (!process.env.STRIPE_SECRET_KEY) throw new Error("STRIPE_SECRET_KEY missing");

export const stripe = new Stripe(process.env.STRIPE_SECRET_KEY, {
  apiVersion: "2026-02-25.clover", // pinned deliberately, see Chapter 15
  appInfo: { name: "reference-billing", version: process.env.RELEASE_SHA ?? "dev" },
  timeout: 20_000,
  maxNetworkRetries: 2,
});

// The single source of truth for what things cost.
export const CATALOGUE = {
  starter_monthly: { priceId: process.env.PRICE_STARTER_MONTHLY!, mode: "subscription" },
  pro_monthly:     { priceId: process.env.PRICE_PRO_MONTHLY!,   mode: "subscription" },
  pro_annual:      { priceId: process.env.PRICE_PRO_ANNUAL!,    mode: "subscription" },
} as const;

export type PlanKey = keyof typeof CATALOGUE;
export const isPlanKey = (v: unknown): v is PlanKey =>
  typeof v === "string" && v in CATALOGUE;
```

D.3 Creating the session

```
// app/api/checkout/route.ts
import { stripe, CATALOGUE, isPlanKey } from "@lib/stripe";
import { requireUser } from "@lib/auth";

export async function POST(req: Request) {
  const user = await requireUser(req);
  if (!user) return new Response("Unauthorised", { status: 401 });

  const body = await req.json().catch(() => ({}));
  if (!isPlanKey(body.plan)) return new Response("Unknown plan", { status: 400 });

  const entry = CATALOGUE[body.plan];

  if (entry.mode === "subscription" && await hasActiveSubscription(user.id)) {
    return new Response("Already subscribed", { status: 409 });
  }

  const customerId = await getOrCreateStripeCustomer(user);
  const bucket = Math.floor(Date.now() / 60_000); // one-minute window

  const session = await stripe.checkout.sessions.create({
    mode: entry.mode,
    customer: customerId,
    line_items: [{ price: entry.priceId, quantity: 1 }],
    client_reference_id: user.id,
    metadata: { userId: user.id, plan: body.plan },
    success_url: `${process.env.BASE_URL}/billing/return?session_id={CHECKOUT_SESSION_ID}`,
    cancel_url: `${process.env.BASE_URL}/pricing`,
    automatic_tax: { enabled: true },
  }, {
    idempotencyKey: `checkout:${user.id}:${body.plan}:${bucket}`,
  });

  return Response.json({ url: session.url });
}
```

D.4 The webhook endpoint

```
// app/api/webhooks/stripe/route.ts
import Stripe from "stripe";
import { stripe } from "@lib/stripe";
import { enqueue } from "@lib/queue";

const SECRET = process.env.STRIPE_WEBHOOK_SECRET!;

const HANDLED = new Set([
  "checkout.session.completed",
  "checkout.session.async_payment_succeeded",
  "checkout.session.async_payment_failed",
  "customer.subscription.created",
  "customer.subscription.updated",
  "customer.subscription.deleted",
  "invoice.paid",
  "invoice.payment_failed",
  "charge.refunded",
  "charge.dispute.created",
  "radar.early_fraud_warning.created",
]);

export async function POST(req: Request) {
  const signature = req.headers.get("stripe-signature");
  if (!signature) return new Response("Missing signature", { status: 400 });

  const raw = await req.text(); // raw body, never req.json()

  let event: Stripe.Event;
  try {
    event = stripe.webhooks.constructEvent(raw, signature, SECRET);
  } catch {
    return new Response("Invalid signature", { status: 400 });
  }

  if (!HANDLED.has(event.type)) {
    return new Response(null, { status: 200 }); // acknowledged, ignored
  }

  try {
    await enqueue({ eventId: event.id, type: event.type });
  } catch {
    return new Response("Queue unavailable", { status: 500 }); // ask for retry
  }

  return new Response(null, { status: 200 });
}
```

D.5 The consumer

```

// workers/stripe-events.ts
import { stripe } from "@lib/stripe";
import { db } from "@lib/db";
import { fulfilCheckout } from "@lib/fulfilment";

export async function consume(job: { eventId: string }) {
  // Claim atomically. A second delivery finds the row already present.
  const claim = await db.query(
    `insert into stripe_events (id, type)
     select $1, $2
     on conflict (id) do nothing
     returning id`,
    [job.eventId, job.type],
  );
  if (claim.rowCount === 0) return;

  // Re-read the event from Stripe rather than trusting the queued copy.
  const event = await stripe.events.retrieve(job.eventId);

  try {
    await route(event);
    await db.query(
      `update stripe_events set processed_at = now() where id = $1`,
      [job.eventId]);
  } catch (err) {
    await db.query(
      `update stripe_events set attempts = attempts + 1, last_error = $2
       where id = $1`, [job.eventId, String(err)]);
    throw err; // queue retries
  }
}

async function route(event: Stripe.Event) {
  switch (event.type) {
    case "checkout.session.completed":
    case "checkout.session.async_payment_succeeded":
      return fulfilCheckout((event.data.object as Stripe.Checkout.Session).id);

    case "checkout.session.async_payment_failed":
      return recordFailedPayment((event.data.object as Stripe.Checkout.Session).id);

    case "customer.subscription.created":
    case "customer.subscription.updated":
    case "customer.subscription.deleted":
      return upsertSubscription(
        event.data.object as Stripe.Subscription,
        new Date(event.created * 1000),
      );

    case "invoice.payment_failed":
      return startDunning(event.data.object as Stripe.Invoice);

    case "charge.refunded":
      return revokeForCharge((event.data.object as Stripe.Charge).id);

    case "charge.dispute.created":
      return handleDispute(event.data.object as Stripe.Dispute);

    case "radar.early_fraud_warning.created":
      return refundProactively(event.data.object as any);

    default:
  }
}

```

```
    return;  
  }  
}
```

D.6 Fulfilment

```
// lib/fulfilment.ts  
export async function fulfilCheckout(sessionId: string): Promise<void> {  
  const session = await stripe.checkout.sessions.retrieve(sessionId, {  
    expand: ["line_items", "payment_intent"],  
  });  
  
  const paid = session.payment_status === "paid"  
    || session.payment_status === "no_payment_required";  
  if (session.status !== "complete" || !paid) return;  
  
  const userId = session.client_reference_id ?? session.metadata?.userId;  
  if (!userId) throw new Error(`Unresolvable session ${sessionId}`);  
  
  const claim = await db.query(  
    `insert into fulfilments (checkout_session_id, user_id)  
    values ($1, $2)  
    on conflict (checkout_session_id) do nothing  
    returning id`,  
    [sessionId, userId],  
  );  
  if (claim.rowCount === 0) return; // already handled  
  
  try {  
    await grantEntitlements(userId, session);  
    await captureEvidence(userId, session);  
    await db.query(  
      `update fulfilments set status = 'done', completed_at = now()  
      where checkout_session_id = $1`, [sessionId]);  
    await sendReceipt(userId, session);  
  } catch (err) {  
    await db.query(  
      `update fulfilments set status = 'failed', last_error = $2  
      where checkout_session_id = $1`, [sessionId, String(err)]);  
    throw err;  
  }  
}
```

D.7 The subscription mirror

```
// lib/subscriptions.ts
const GRANTS_ACCESS = new Set(["active", "trialing", "past_due"]);

export function hasAccess(status: string): boolean {
  return GRANTS_ACCESS.has(status); // one place, used everywhere
}

export async function upsertSubscription(sub: Stripe.Subscription, eventAt: Date) {
  const userId = await resolveUserByCustomer(sub.customer as string);
  if (!userId) throw new Error(`Unknown customer ${sub.customer}`);

  await db.query(
    `insert into subscriptions
      (id, user_id, stripe_customer_id, status, price_id,
       current_period_end, cancel_at_period_end, last_event_created_at)
    values ($1,$2,$3,$4,$5,to_timestamp($6),$7,$8)
    on conflict (id) do update set
      status           = excluded.status,
      price_id         = excluded.price_id,
      current_period_end = excluded.current_period_end,
      cancel_at_period_end = excluded.cancel_at_period_end,
      updated_at       = now(),
      last_event_created_at = excluded.last_event_created_at
    where subscriptions.last_event_created_at < excluded.last_event_created_at`,
    [
      sub.id, userId, sub.customer, sub.status,
      sub.items.data[0]?.price.id,
      sub.current_period_end, sub.cancel_at_period_end, eventAt,
    ],
  );
}
```

D.8 The return page

```
// app/billing/return/page.tsx
export default async function ReturnPage({ searchParams }) {
  const user = await requireUser();
  const sessionId = searchParams.session_id;
  if (!sessionId) redirect("/pricing");

  const session = await stripe.checkout.sessions.retrieve(sessionId);

  // The session must belong to this user.
  if (session.client_reference_id !== user.id) redirect("/billing");

  // Idempotent, so calling it here costs nothing and makes the
  // experience feel immediate. The webhook remains authoritative.
  await fulfilCheckout(sessionId);

  return <ThankYou email={session.customer_details?.email} />;
}
```

D.9 Reconciliation

```
// jobs/reconcile.ts, scheduled nightly
export async function reconcile(sinceHours = 48) {
  const since = Math.floor(Date.now() / 1000) - sinceHours * 3600;
  let drift = 0;

  for await (const sub of stripe.subscriptions.list({ created: { gte: since }, limit: 100 })) {
    const local = await getLocalSubscription(sub.id);
    if (!local || local.status !== sub.status) {
      await upsertSubscription(sub, new Date());
      drift++;
      logger.warn({ subscriptionId: sub.id }, "reconciliation corrected drift");
    }
  }

  // The alert that matters most: money in, nothing delivered.
  const orphaned = await db.query(
    `select cs.id from paid_sessions cs
     left join fulfilments f on f.checkout_session_id = cs.id
     where f.id is null and cs.created_at < now() - interval '30 minutes'`;
  if (orphaned.rowCount > 0) await pageOnCall("payments without fulfilment", orphaned.rows);

  metrics.gauge("billing.reconciliation.drift", drift);
}
```

CONTEXT

This is deliberately minimal. It omits tax handling, proration, upgrade and downgrade paths, multi-currency, and the customer portal. What it does not omit is any of the security properties, because those are the parts that must be present in the first version rather than added later.

APPENDIX E

Glossary

Terms used throughout, defined as they are used in this book.

Acquirer

The financial institution that holds the merchant account and settles funds. Stripe works with acquirers on your behalf.

Application fee

The amount a Connect platform retains from a charge processed for a connected account.

Authorisation

The issuer's approval to hold funds. Distinct from capture, which actually moves them.

Blast radius

The extent of damage a single compromised credential permits. Reducing it is the purpose of restricted keys.

Capture

Converting an authorisation into an actual charge. Can be immediate or deferred.

Cardholder data

The primary account number and associated elements. Its presence in a system is what puts that system in PCI scope.

Card testing

Automated probing of stolen card numbers against a merchant to discover which remain valid.

Chargeback

The forced reversal of a settled transaction initiated through the cardholder's bank. Used interchangeably with dispute in this book.

Client secret

A scoped credential that permits a browser to confirm one specific `PaymentIntent` or `SetupIntent`.

Connected account

An account belonging to a seller or service provider on a Connect platform.

Destination charge

A charge created on the platform account and automatically transferred to a connected account.

Dispute

A cardholder's formal claim through their issuer that a charge was incorrect.

Dunning

The sequence of retries and communications following a failed recurring payment.

Early fraud warning

A signal from the issuer that a charge is likely fraudulent, arriving before a dispute is filed.

Elements

Stripe's composable UI components, hosted in Stripe-controlled iframes.

E-skimming

Theft of card data by modifying a script on the merchant's payment page. Also called Magecart.

Exemption

A regulatory basis for skipping strong customer authentication. Requested by the merchant, granted by the issuer.

Fulfilment

Granting what was paid for: access, goods, credits, provisioning.

Idempotency key

A client-supplied identifier that makes a repeated request return the original result rather than executing again.

Issuer

The bank that issued the customer's card and decides whether to authorise a transaction.

Liability shift

The transfer of responsibility for a fraudulent-transaction chargeback from merchant to issuer following successful authentication.

Merchant of record

The legal entity that appears as the seller, bears the transaction liability and carries the tax obligation.

MIT

Merchant initiated transaction. A charge made without the customer present, under a previously agreed mandate.

Off-session

A charge made when the customer is not present to authenticate.

PAN

Primary account number. The card number itself.

PaymentIntent

The object tracking a payment through its lifecycle, including authentication and retries.

PCI DSS

The card industry's data security standard, enforced contractually through acquirers.

PSD2

The second EU Payment Services Directive, which introduced strong customer authentication.

Radar

Stripe's fraud detection system, scoring payments against platform-wide signals.

Reserve

Funds withheld from payout to cover future liability such as disputes and refunds.

Restricted key

An API key granted specific permissions on specific resources rather than full account authority.

SAQ

Self-Assessment Questionnaire. The PCI compliance form appropriate to your integration.

SCA

Strong Customer Authentication. Two independent factors, required for most electronic payments in the EEA and UK.

SetupIntent

The object used to collect and authenticate a payment method for future use without charging it now.

Signing secret

The per-endpoint key used to verify that an inbound webhook genuinely originated from Stripe.

SRI

Subresource integrity. A hash attached to a script tag so the browser refuses modified files.

Statement descriptor

The text a customer sees on their bank statement. A leading cause of unrecognised-charge disputes.

3D Secure

The protocol that delivers cardholder authentication and, with it, the liability shift.

Tokenisation

Replacing card data with a reference that carries no value if stolen.

TRA

Transaction Risk Analysis. An SCA exemption available to acquirers with low fraud rates.

Webhook

An HTTP callback delivering event notifications from Stripe to your server.



APPENDIX F

Sources

Every normative claim in this book traces to one of these. They are also the places to check when a figure here has aged.

Stripe documentation

- **Security at Stripe.** Platform security posture, encryption, TLS enforcement, PCI Level 1 service provider status, vulnerability disclosure. docs.stripe.com/security
- **Integration security guide.** Merchant-side obligations, PCI scope by integration type, key handling, HTTPS requirements. docs.stripe.com/security/guide
- **API keys and best practices.** Key types, restricted keys, rotation, storage, incident response. docs.stripe.com/keys-best-practices
- **Webhooks.** Signature verification, raw body handling, retry behaviour, ordering guarantees, endpoint configuration. docs.stripe.com/webhooks
- **Fulfil orders with Checkout.** The idempotent fulfilment pattern and the dual-caller structure. docs.stripe.com/checkout/fulfillment
- **Rate limits.** Per-account and per-endpoint limits, live and sandbox differences, retry guidance. docs.stripe.com/rate-limits
- **Idempotent requests.** Key semantics, retention window, conflict behaviour. docs.stripe.com/api/idempotent_requests
- **Error handling.** Error types, codes, and which are safely retryable. docs.stripe.com/error-handling
- **Strong Customer Authentication and 3D Secure.** Regulatory scope, authentication flows, exemption handling. docs.stripe.com/strong-customer-authentication
- **SCA exemptions.** Low value, TRA thresholds, recurring and merchant initiated transactions. docs.stripe.com/payments/3d-secure/authentication-flow
- **Radar.** Risk scores, risk levels, rules, card testing protections, reviews. docs.stripe.com/radar
- **Disputes and fraud.** Reason codes, evidence submission, deadlines, early fraud warnings. docs.stripe.com/disputes
- **Connect.** Account types, controller properties, charge types, liability configuration. docs.stripe.com/connect
- **API versioning and upgrades.** Release trains, account default versions, per-request and per-endpoint overrides. docs.stripe.com/api/versioning and docs.stripe.com/upgrades
- **Testing.** Test cards, simulated declines, authentication scenarios, CLI event triggers. docs.stripe.com/testing
- **Going live checklist.** Pre-launch verification across account, integration and operations. docs.stripe.com/get-started/checklist/go-live
- **Content Security Policy guidance.** Required origins for Stripe.js, Elements and Checkout. docs.stripe.com/security/guide
- **Prohibited and restricted businesses.** The authoritative category list. stripe.com/legal/restricted-businesses
- **Global availability.** Supported countries and account jurisdiction rules. stripe.com/global

Standards and regulation

- **PCI Security Standards Council, PCI DSS v4.0.1.** The full standard, including requirements 6.4.3 and 11.6.1 and their 31 March 2025 effective date. pcisecuritystandards.org
- **PCI SSC self-assessment questionnaires.** SAQ A, SAQ A-EP and SAQ D, with their eligibility criteria.
- **PCI SSC information supplements on payment page security and script management.** Practical guidance on satisfying the client-side requirements.
- **Directive (EU) 2015/2366 (PSD2)** and **Commission Delegated Regulation (EU) 2018/389** on regulatory technical standards for strong customer authentication, including the exemption thresholds in Figure 5.1.
- **UK Financial Conduct Authority Payment Services Regulations** and associated approach documents, for the retained UK regime.
- **EMVCo 3-D Secure specification, version 2.** Authentication data flows and challenge mechanics.

Security research and field data

- **Published scans of publicly reachable Stripe webhook endpoints,** which found that roughly a quarter of approximately six thousand applications examined performed no signature verification. The basis for Figures 1.2 and 1.3.
- **Documented vulnerabilities in third party Stripe webhook handling libraries,** including signature verification bypasses disclosed during 2025 and 2026. A reminder that using a library is not the same as using it correctly.
- **Public reporting on Magecart and e-skimming campaigns,** which motivated the client-side requirements added in PCI DSS 4.0.
- **Stripe's published engineering guidance for building on the platform,** including their internal security best practices for integrations.
- **OWASP Application Security Verification Standard,** for the general controls referenced in Chapters 6, 7 and 11.

CONTEXT

Where this book gives a specific number (a rate limit, an exemption threshold, an API version string, a risk score band), it reflects the source as published at the time of writing. Payment platforms and standards bodies both revise on their own schedules. Before any figure here informs a commercial or compliance decision, check it against the primary source above.

COLOPHON

Set in Source Serif 4 for text, Inter for headings and tabular matter, and JetBrains Mono for code. Figures generated from the underlying data rather than drawn. Composed as structured HTML and rendered to PDF.

A CLOSING NOTE

The controls in this book are not difficult. Verifying a signature is one function call. Resolving a price server side is a lookup. Making fulfilment idempotent is a unique constraint. What makes payment integrations fail is not that these things are hard, but that nothing in the ordinary course of building and testing a product ever forces you to do them. The happy path works without any of them. That is the whole problem, and the reason a checklist is worth more here than in almost any other part of engineering.

THE MONEY PATH

Ismaël Ladjohounlou · First edition, 2026